

MINIMUM SPANNING TREE (MST)

1. Overview

A **Minimum Spanning Tree (MST)** of a weighted, connected, undirected graph is a subset of edges that:

- Connects **all vertices** together (spanning tree)
- Has **no cycles** (tree property)
- Minimizes the **total edge weight** among all such spanning trees

Think of it as: *"Connect all points with the smallest total cost, without creating loops."*

If the graph has V vertices, an MST always has exactly $V-1$ edges.

2. Intuition

Simple Explanation

Imagine you're a telecom company laying fiber optic cables to connect V cities. Each city-to-city connection has a cost. You want to connect every city so they can all communicate, but you want to spend the **least amount of money** possible.

The MST is your answer: the cheapest set of cables that connects all cities.

Analogy

Road network analogy: You need to build roads so that every village is reachable from every other village. Building roads costs money per kilometer. You want the minimum total cost road network. You'd never build a road that creates a cycle (a loop) because you could remove that road and still have all villages connected — that's wasteful.

Why It Works

The MST problem exhibits **optimal substructure** and the **greedy choice property**:

- **Optimal substructure:** Any sub-tree of an MST is itself an MST of the vertices it connects.
- **Greedy choice:** The smallest-weight edge that connects two different components (or a vertex to the growing tree) is always safe to include.

Two classic greedy algorithms exploit this: **Kruskal's** (edge-based) and **Prim's** (vertex-based).

3. When to Use It

Use MST algorithms when:

- The problem asks for the **minimum cost to connect all nodes** in a graph.
- The problem asks for a **minimum total weight** spanning subgraph.
- The problem involves **connecting points** with the least cost.

- You need to find a **subset of edges** that keeps the graph connected with minimum total weight.

Common Trigger Phrases

- "Minimum cost to connect all cities"
- "Network design with minimum cost"
- "Connect all points with minimum total edge length"
- "Find a spanning tree with minimum sum of weights"
- "Cheapest way to connect all computers/nodes/houses"
- "Minimum total length of cables/wires/roads"
- "Remove minimum weight edges to disconnect" (complement: MST gives maximum weight to keep connected)

4. When Not to Use It

Situation	Reason
Need shortest path between two nodes	Use Dijkstra / Bellman-Ford. MST minimizes total cost of connecting all nodes, not distance between two specific nodes.
Graph is directed	MST is defined for undirected graphs. For directed graphs, use Arborescence / Minimum Spanning Arborescence (Edmonds' algorithm).
Graph is unweighted	Any spanning tree works. BFS tree is sufficient.
Need maximum spanning tree	Negate weights or invert comparator, but use the same algorithm.
Graph is disconnected	MST doesn't exist. Use Minimum Spanning Forest (run Kruskal/Prim, it will stop with $< V-1$ edges).
Graph is very dense and you need fast implementation	Prim with adjacency matrix is $O(V^2)$. Kruskal with sorting is $O(E \log E)$. Both are fine.
Need to handle dynamic edge insertions/deletions	Dynamic MST algorithms exist but are complex. For simple cases, re-run Kruskal/Prim.
Very large graphs ($E > 10^7$)	Sorting all edges may be memory-heavy. Prim ($O(E \log V)$) with adjacency list may be better.

5. Core Concepts

5.1 Spanning Tree

A subgraph that is a tree (connected, acyclic) and includes **all vertices** of the original graph.

- Why it matters:** MST is just a spanning tree with minimum total weight.
- Property:** A spanning tree of V vertices always has exactly $V-1$ edges.

5.2 Cut Property

If you partition vertices into two sets, the **minimum-weight edge** crossing the cut belongs to **some** MST.

- **Why it matters:** This is the theoretical foundation of both Kruskal's and Prim's algorithms.
- **Example:** If vertices are split into {A, B} and {C, D}, and the smallest edge connecting the two groups is (B, C) with weight 2, then (B, C) must be in some MST.

5.3 Cycle Property

If you have a cycle, the **maximum-weight edge** in that cycle is **not** in any MST.

- **Why it matters:** This is the basis for reverse-delete algorithms and helps reason about edge selection.
- **Example:** In a cycle A-B-C-A with edges (A,B)=5, (B,C)=3, (C,A)=4, the edge (A,B)=5 is the heaviest and will never be in an MST.

5.4 Disjoint Set Union (DSU) / Union-Find

A data structure that tracks which vertices belong to which component. Supports:

- `find(u)`: Find which component `u` belongs to.
- `union(u, v)`: Merge the components of `u` and `v`.
- **Why it matters:** Kruskal's algorithm uses DSU to efficiently check if adding an edge creates a cycle.

5.5 Priority Queue / Min-Heap

A data structure that always gives the smallest element.

- **Why it matters:** Prim's algorithm uses a min-heap to efficiently find the smallest edge connecting the tree to a new vertex.

6. Step-by-Step Algorithm

Kruskal's Algorithm

1. Sort all edges by weight (ascending).
2. Initialize DSU where each vertex is its own component.
3. Initialize `mst_weight = 0`, `edges_taken = 0`.
4. For each edge `(u, v, w)` in sorted order:
 - If `find(u) != find(v)` (they are in different components):
 - Add this edge to MST.
 - `mst_weight += w`
 - `union(u, v)`
 - `edges_taken += 1`
 - If `edges_taken == V-1`, stop.
5. Output `mst_weight`.

Prim's Algorithm

1. Start with any vertex (say vertex 0).

2. Maintain:

- `vis[]` — visited vertices (in MST).
- min-heap of `(weight, vertex)` — edges crossing the cut.
- `mst_weight = 0`

3. Push `(0, 0)` — cost 0 to start at vertex 0.

4. While heap is not empty:

- Pop `(w, u)`.
- If `vis[u]` is true, continue.
- Mark `vis[u] = true, mst_weight += w`.
- For each neighbor `v` of `u` with edge weight `w2`:
 - If not `vis[v]`, push `(w2, v)`.

5. Output `mst_weight`.

7. Dry Run

Kruskal's Algorithm — Dry Run

Input Graph:

```
Vertices: 0, 1, 2, 3, 4
Edges:
0-1: 2
0-3: 6
1-2: 3
1-3: 8
1-4: 5
2-4: 7
3-4: 9
```

Step 1: Sort edges by weight.

Edge	Weight
0-1	2
1-2	3
1-4	5
0-3	6
2-4	7
1-3	8
3-4	9

Step 2: DSU initialization. Each vertex is its own component: `{0}, {1}, {2}, {3}, {4}`.

Step 3: Process edges in order.

Edge	Weight	find(u)!=find(v)?	Action	Components after	edges_taken
0-1	2	Yes (0≠1)	Add	{0,1}, {2}, {3}, {4}	1
1-2	3	Yes (1≠2)	Add	{0,1,2}, {3}, {4}	2
1-4	5	Yes (1≠4)	Add	{0,1,2,4}, {3}	3
0-3	6	Yes (0≠3)	Add	{0,1,2,3,4}	4
2-4	7	No (same component)	Skip	—	4
1-3	8	(already V-1=4 edges)	Stop	—	4

MST Edges: (0-1, 2), (1-2, 3), (1-4, 5), (0-3, 6) **Total MST Weight:** 2 + 3 + 5 + 6 = **16**

Prim's Algorithm — Dry Run

Same graph. Start at vertex 0.

Step	u (popped)	w	vis[]	mst_weight	Heap state (pushed)
0	—	—	{0}	0	(0,0)
1	0	0	{0}	0	(2,1), (6,3)
2	1	2	{0,1}	2	(3,2), (5,4), (8,1→skip), (6,3)
3	2	3	{0,1,2}	5	(5,4), (7,2→skip), (6,3)
4	4	5	{0,1,2,4}	10	(6,3), (9,4→skip)
5	3	6	{0,1,2,3,4}	16	—

Total MST Weight: 16 ✓

8. C++ Implementation

Kruskal's Algorithm

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u, v, w;
};

class DSU {
public:
    vector<int> parent, rank;
    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
    }
};
```

```

        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) swap(rx, ry);
        parent[ry] = rx;
        if (rank[rx] == rank[ry]) rank[rx]++;
    }
};

// Kruskal: returns total MST weight and the edges taken
pair<int, vector<Edge>> kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);
    vector<Edge> mst;
    int totalWeight = 0;

    for (auto& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            mst.push_back(e);
            totalWeight += e.w;
            if ((int)mst.size() == n - 1) break;
        }
    }
    return {totalWeight, mst};
}

int main() {
    int n = 5; // vertices
    vector<Edge> edges = {
        {0, 1, 2}, {0, 3, 6}, {1, 2, 3},
        {1, 3, 8}, {1, 4, 5}, {2, 4, 7}, {3, 4, 9}
    };

    auto [weight, mst] = kruskal(n, edges);

    cout << "Total MST Weight: " << weight << "\n";
    cout << "Edges:\n";
    for (auto& e : mst)
        cout << e.u << " - " << e.v << " : " << e.w << "\n";
    return 0;
}

```

Prim's Algorithm

```
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

// Prim: returns total MST weight (using adjacency list)
int prim(int n, vector<vector<pii>>& adj) {
    vector<bool> vis(n, false);
    priority_queue<pii, vector<pii>, greater<pii>> pq; // min-heap of (weight,
vertex)
    int mstWeight = 0;
    int edgesTaken = 0;

    pq.push({0, 0}); // start from vertex 0

    while (!pq.empty() && edgesTaken < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        mstWeight += w;
        edgesTaken++;

        for (auto& [v, wt] : adj[u]) {
            if (!vis[v]) pq.push({wt, v});
        }
    }
    return mstWeight;
}

int main() {
    int n = 5;
    vector<vector<pii>> adj(n);
    vector<tuple<int,int,int>> edges = {
        {0,1,2}, {0,3,6}, {1,2,3}, {1,3,8}, {1,4,5}, {2,4,7}, {3,4,9}
    };
    for (auto& [u,v,w] : edges) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    cout << "Total MST Weight (Prim): " << prim(n, adj) << "\n";
    return 0;
}
```

9. Python Implementation

Kruskal's Algorithm

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def unite(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1

def kruskal(n, edges):
    # edges: list of (u, v, w)
    edges.sort(key=lambda e: e[2])
    dsu = DSU(n)
    mst = []
    total_weight = 0

    for u, v, w in edges:
        if dsu.find(u) != dsu.find(v):
            dsu.unite(u, v)
            mst.append((u, v, w))
            total_weight += w
            if len(mst) == n - 1:
                break

    return total_weight, mst

# Example usage
n = 5
edges = [
    (0, 1, 2), (0, 3, 6), (1, 2, 3),
    (1, 3, 8), (1, 4, 5), (2, 4, 7), (3, 4, 9)
]
weight, mst = kruskal(n, edges)
print(f"Total MST Weight: {weight}")
print("Edges:", mst)

```

Prim's Algorithm

```

import heapq

def prim(n, adj):
    # adj: list of list of (neighbor, weight)
    vis = [False] * n
    pq = [(0, 0)] # (weight, vertex)
    mst_weight = 0
    edges_taken = 0

    while pq and edges_taken < n:
        w, u = heapq.heappop(pq)
        if vis[u]:
            continue
        vis[u] = True
        mst_weight += w
        edges_taken += 1

        for v, wt in adj[u]:
            if not vis[v]:
                heapq.heappush(pq, (wt, v))

    return mst_weight

# Example usage
n = 5
edges = [(0, 1, 2), (0, 3, 6), (1, 2, 3), (1, 3, 8), (1, 4, 5), (2, 4, 7), (3, 4, 9)]
adj = [[] for _ in range(n)]
for u, v, w in edges:
    adj[u].append((v, w))
    adj[v].append((u, w))

print(f"Total MST Weight (Prim): {prim(n, adj)}")

```

10. Code Explanation

Kruskal Implementation — Key Parts

DSU Class:

- `parent[]`: Stores the root/representative of each component.
- `rank[]`: Keeps tree depth small during union.
- `find(x)`: Uses **path compression** — recursively finds root and flattens the tree. Amortized $O(\alpha(n))$.
- `unite(x, y)`: Attaches the smaller rank tree under the larger rank tree to keep depth minimal.

Sorting:

- `sort(edges.begin(), edges.end(), ...)` — the bottleneck. Sorting all edges by weight takes $O(E \log E)$.

Main Loop:

- For each edge (in increasing weight), check if `u` and `v` are in different components.
- If yes, union them, add to MST, accumulate weight.
- Early stop when `edges_taken == n-1` (MST is complete).

Prim Implementation — Key Parts

Adjacency List:

- `adj[u] = list of (v, w)` — stores neighbors and edge weights.
- Since graph is undirected, each edge is added twice.

Priority Queue:

- Min-heap of `(weight, vertex)`.
- `greater<pii>` makes it a min-heap.

Main Loop:

- Pop the smallest weight edge crossing the cut.
- Skip if vertex already visited (that edge was already considered).
- Mark visited, add weight, push all neighbors' edges.
- Stop when `edges_taken == n` (all vertices visited).

Key difference from Dijkstra: Prim's does not update distances of existing vertices in the heap. It just pushes new entries. Visited check handles duplicates.

11. Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Notes
Kruskal (sorting)	$O(E \log E) = O(E \log V)$	$O(V + E)$	Sorting dominates. DSU operations are nearly $O(1)$.
Kruskal (counting sort for small weights)	$O(E + V)$	$O(V + E)$	If weights are small integers, use counting sort.
Prim (adj list + binary heap)	$O(E \log V)$	$O(V + E)$	Standard for sparse graphs.
Prim (adjacency matrix)	$O(V^2)$	$O(V^2)$	Better for dense graphs ($E \approx V^2$).
Prim (Fibonacci heap)	$O(E + V \log V)$	$O(V + E)$	Theoretical improvement, rarely used in practice.

Detailed Breakdown

	Preprocessing	Main Algorithm	Query
Kruskal	$O(E \log E)$ sorting	$O(E \alpha(V))$ for DSU operations	N/A
Prim	$O(V)$ initialize arrays	$O(E \log V)$ heap operations	N/A

Best case: Sparse graph ($E \approx V$) $\rightarrow$ Kruskal $O(V \log V)$ or Prim $O(V \log V)$. **Worst case:** Dense graph ($E \approx V^2$) $\rightarrow$ Prim with adjacency matrix $O(V^2)$ is better than Kruskal $O(V^2 \log V)$.

12. Common Patterns

Pattern 1: Minimum Cost to Connect All Nodes

How to identify: Problem asks for minimum cost/effort to connect all given points/nodes/cities.

Approach: Direct MST. Build graph from given connections, run Kruskal or Prim.

Examples: LeetCode 1135 (Connecting Cities With Minimum Cost), GFG — Minimum Cost to Connect All Cities.

Pattern 2: Maximum Spanning Tree

How to identify: Problem asks for maximum cost to connect all nodes, or "remove minimum edges to keep connected" (complement).

Approach: Sort edges in descending order, or negate weights and run standard MST.

Example: CSES — Maximum Spanning Tree.

Pattern 3: Second Best / K-th Best MST

How to identify: "Find second cheapest way to connect", "alternative minimum spanning tree".

Approach: Find MST. For each edge in MST, temporarily remove it, run MST again. Take minimum of these. $O(E \log E + E * V \alpha(V))$.

Example: CSES — Second Minimum Spanning Tree.

Pattern 4: Minimum Steiner Tree (subset of vertices)

How to identify: "Connect a subset of vertices with minimum cost".

Approach: This is NP-hard. For small k (≤ 15), use DP over subsets (DP with bitmask + shortest paths).

Example: LeetCode — Minimum Cost to Connect Points (specific case when all vertices must be connected).

Pattern 5: Critical Edges / Bridges in MST

How to identify: "Find edges that MUST be in every MST", "edges whose removal increases MST cost".

Approach: Run Kruskal. For edges with equal weight, if they connect same components, they are not critical. If removing an edge forces a higher-weight edge, it's critical.

Example: LeetCode 1489 (Find Critical and Pseudo-Critical Edges in MST).

Pattern 6: Minimum Spanning Forest

How to identify: "Connect all nodes, but you have k initial components", "connect with k cables".

Approach: Run Kruskal until you have k components instead of 1 (stop when `edges_taken == n - k`).

Example: LeetCode — Minimum Cost to Connect Cities (with existing highways).

13. Common Mistakes

Mistake	Explanation
Forgetting undirected graph	Add edges in both directions for Prim. Kruskal only needs edges once.
Not using path compression in DSU	Without it, DSU becomes $O(V)$ per find, making Kruskal $O(EV)$.
Not checking visited in Prim	Without <code>vis[]</code> , the same vertex can be added multiple times, giving wrong weight.
Overflow in total weight	Sum of weights can exceed <code>int</code> . Use <code>long long</code> in C++ / Python's <code>int</code> is arbitrary precision.
Not handling disconnected graphs	Kruskal stops with $< V-1$ edges. Prim might not visit all vertices. Check <code>edges_taken == n-1</code> .
Wrong comparator in priority_queue	<code>priority_queue<int></code> is max-heap by default. Use <code>greater<pii></code> for min-heap.
Sorting edges incorrectly	Must sort by weight ascending. A single wrong comparator can give wrong answer.
Modifying DSU while iterating	In some languages, iterating while modifying can cause issues. C++ is fine.
Assuming MST is unique	If multiple edges have the same weight, there can be multiple MSTs.
Forgetting 0-index vs 1-index	Input might use 1-indexed vertices. Adjust accordingly.

14. Edge Cases

Edge Case	What to Expect
Empty graph (n=0)	No edges, MST weight = 0.
Single vertex (n=1)	No edges needed. MST weight = 0.
Two vertices, one edge	That edge is the MST.
Disconnected graph	Kruskal returns $< V-1$ edges. Prim won't visit all vertices. Detect and handle.

Edge Case	What to Expect
All edges have same weight	Many MSTs exist. Any algorithm will find one.
Graph already a tree	The whole graph is the MST.
Complete graph	Prim's $O(V^2)$ matrix approach is simpler. Kruskal will sort V^2 edges.
Negative weights	MST algorithms work fine with negative weights.
Self-loops	A self-loop (u, u) can never be in an MST. Ignore them.
Parallel edges	Only the smallest weight edge between two vertices matters.
Large weights (up to 10^9)	Use <code>long long</code> for total weight.
Multiple components	Check if graph is connected before claiming MST exists.

15. Variations

15.1 Maximum Spanning Tree

What changes: Sort edges in descending order instead of ascending.

When used: When you want the maximum total cost spanning tree.

Importance: Low. Rarely asked, but easy to adapt.

15.2 Second Best MST

What changes: Find MST, then for each MST edge, remove it and find MST again. Take the minimum among these.

When used: Problems asking for "alternative" or "second cheapest" spanning tree.

Importance: Medium. Appears in some contests.

15.3 Minimum Bottleneck Spanning Tree (MBST)

What changes: A spanning tree that minimizes the maximum edge weight. Every MST is an MBST, but not vice versa.

When used: Problems where you want to minimize the worst individual edge cost.

Importance: Low. Mostly theoretical.

15.4 Euclidean MST

What changes: Points are on a 2D plane. Edge weight = Euclidean distance between points. Use Delaunay triangulation for $O(V \log V)$.

When used: Connecting points by their geometric distance.

Importance: Medium. Appears in some CP problems.

15.5 Dynamic MST

What changes: Edges are inserted/deleted over time. Use specialized data structures (Link-Cut Tree, Top Trees, or recompute from scratch).

When used: Problems where graph changes and you need MST after each change.

Importance: High for CP. Uses Link-Cut Tree or offline methods (divide and conquer + DSU rollback).

15.6 Minimum Steiner Tree

What changes: Only a subset of vertices (terminals) need to be connected. You can use additional intermediate vertices.

When used: "Connect these specific points with minimum cost, can use other points as relay."

Importance: Low for interviews. NP-hard. DP over subsets for small k.

16. Related Algorithms / Data Structures

Algorithm/DS	How It's Connected	When to Choose
DSU	Kruskal's backbone. Tracks components.	Always use with Kruskal.
Dijkstra's Algorithm	Also uses a priority queue, but for shortest paths.	MST vs Shortest Path: MST connects all nodes cheaply; Dijkstra finds cheapest path from source to every other node.
Prim vs Dijkstra	Code is almost identical. Dijkstra uses distance from source; Prim uses distance to the tree.	Prim: $\text{dist}[v] = \text{edge_weight}$ (no accumulation). Dijkstra: $\text{dist}[v] = \text{dist}[u] + \text{edge_weight}$.
BFS	For unweighted graphs, BFS gives a spanning tree (not necessarily minimum).	Use BFS if graph is unweighted.
Floyd-Warshall	Can compute all-pairs shortest paths, then build MST from complete graph.	Only if the graph is given as a set of points with distances, not explicit edges.
Link-Cut Tree	Supports dynamic MST (edge insert/delete).	When you need MST under edge updates. Hard to implement in interviews.
Borůvka's Algorithm	Another MST algorithm. Processes all vertices simultaneously.	$O(E \log V)$. Useful for parallel MST. Rarely asked.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Connecting Cities With Minimum Cost	LeetCode 1135	Direct MST (Kruskal)	Easy
Minimum Spanning Tree	GFG	Direct MST (Prim/Kruskal)	Easy

Medium

Problem	Platform	Pattern	Difficulty
Min Cost to Connect All Points	LeetCode 1584	MST on complete graph (Prim $O(V^2)$ works best)	Medium
Find Critical and Pseudo-Critical Edges in MST	LeetCode 1489	Critical edges detection	Hard-Medium
Road Reparation	CSES	Direct MST (Kruskal), check connectivity	Medium

Hard

Problem	Platform	Pattern	Difficulty
Second Minimum Spanning Tree	CSES	Second Best MST	Hard
Dynamic Graph Connectivity	Codeforces	MST with DSU rollback / offline	Hard
Minimum Spanning Tree for Each Edge	Codeforces 609E	MST + LCA for max edge on path	Hard
Rebuilding Roads	AtCoder ABC 270 F	MST with bridge detection	Hard

18. Interview Explanation

"A Minimum Spanning Tree of a weighted, connected, undirected graph is a subset of $V-1$ edges that connects all vertices with the minimum possible total weight.

There are two main algorithms: **Kruskal's** and **Prim's**.

Kruskal's works by sorting all edges by weight and then greedily adding the smallest edge that doesn't create a cycle. To detect cycles efficiently, we use a **Disjoint Set Union** data structure with path compression and union by rank, giving near-constant time per operation. The overall complexity is $O(E \log E)$ due to sorting.

Prim's works by growing a tree from a starting vertex. At each step, it adds the smallest edge that connects the current tree to a vertex outside it. It uses a min-heap for efficiency, giving $O(E \log V)$ complexity.

The key theoretical foundations are the **cut property** — the smallest edge crossing any cut belongs to some MST — and the **cycle property** — the heaviest edge in any cycle never belongs to an MST.

I'd use **Kruskal's** when the edge list is given explicitly, and **Prim's** when the graph is dense (like a complete graph of points with distances). MST problems commonly appear in network design, connecting cities, or minimizing total connection cost."

19. Revision Notes

- **MST:** $V-1$ edges, no cycles, connects all vertices, minimum total weight.
- **Kruskal:** Sort edges $\rightarrow$ DSU $\rightarrow$ add smallest non-cycle edge.
 - Complexity: **$O(E \log E)$**
 - Use when: edges are given explicitly.
- **Prim:** Start from any vertex $\rightarrow$ min-heap $\rightarrow$ add smallest edge to tree.
 - Complexity: **$O(E \log V)$**
 - Use when: graph is dense or adjacency list is natural.
- **DSU:** `find()` with path compression, `unite()` with union by rank. **$O(\alpha(n))$**
- **Disconnected graph:** MST doesn't exist. Check `edges_taken == V-1`.
- **Overflow:** Use `long long` for total weight.
- **Multiple MSTs:** Equal-weight edges can cause multiple MSTs. Some problems exploit this.
- **Second Best MST:** Remove each MST edge, run MST again. $O(E \log E + E * V)$.
- **Critical edges:** Edge that appears in every MST. Remove it, graph becomes disconnected or MST weight increases.
- **Key formula:** MST weight = sum of selected edges.

20. Final Cheat Sheet

MST CHEAT SHEET		
WHEN TO USE:		
• "Minimum cost to connect all nodes" • "Connect all points with minimum total weight" • "Network design with minimum cost"		
MAIN OPERATIONS:		
Kruskal: sort edges $O(E \log E)$ + DSU $O(E \alpha(V))$		
Prim: heap push/pop $O(E \log V)$ + visitor $O(V)$		
COMPLEXITY:		
	Time	Space
Kruskal	$O(E \log E)$	$O(V + E)$
Prim	$O(E \log V)$	$O(V + E)$
Prim (M)	$O(V^2)$	$O(V^2)$
KEY CODE IDEA:		
Kruskal: sort edges $\rightarrow$ for each: if <code>find(u) != find(v)</code> $\rightarrow$ add		
Prim: <code>pq.push({0,0})</code> $\rightarrow$ while: pop, if !vis, add, push		

EDGE CASES:

- $n=0$, $n=1 \rightarrow \text{weight} = 0$
- Disconnected $\rightarrow$ check `edges_taken == n-1`
- Negative weights $\rightarrow$ works fine
- Overflow $\rightarrow$ use `long long`
- Parallel edges $\rightarrow$ keep only min weight
- Self-loops $\rightarrow$ ignore

COMMON TRAPS:

- Forgetting `vis[]` in Prim
- No path compression in DSU
- Wrong comparator in `priority_queue` (max vs min)
- Not handling 0-index vs 1-index
- Assuming MST is unique

DISJOINT SET UNION (DSU) FOR KRUSKAL

1. Overview

Disjoint Set Union (DSU), also called **Union-Find**, is a data structure that tracks a set of elements partitioned into disjoint (non-overlapping) subsets. It supports two operations efficiently:

- `find(x)`: Find which subset x belongs to (return the representative/root).
- `union(x, y)`: Merge the subsets containing x and y .

In the context of Kruskal's algorithm, DSU is used to detect whether adding an edge would create a cycle — if two vertices are already in the same component, adding the edge creates a cycle.

2. Intuition

Simple Explanation

Imagine you have a group of people at a party. Initially, everyone is standing alone. When two people shake hands, they form a group. If two people from the same group shake hands, it's just a cycle inside the group. If two people from different groups shake hands, the groups merge.

DSU answers two questions:

- "Which group is this person in?" $\rightarrow$ `find(x)`
- "Merge these two groups." $\rightarrow$ `union(x, y)`

Analogy

Family tree analogy: Each person initially belongs to their own family. When two families merge (marriage), everyone in the merged family shares the same ancestor. To find if two people are in the

same family, you trace their ancestry to the root. Path compression is like updating everyone's records to point directly to the root ancestor after a lookup.

Why It Works

- **Path compression:** While finding the root, we flatten the tree so that all nodes on the path point directly to the root. This makes future finds $O(1)$ amortized.
- **Union by rank/size:** We always attach the smaller tree under the larger tree. This keeps the tree depth $O(\log n)$, and with path compression, it becomes nearly constant.

The combination of these two optimizations gives **amortized $O(\alpha(n))$** per operation, where $\alpha(n)$ is the inverse Ackermann function — **essentially constant for all practical input sizes**.

3. When to Use It

Use DSU when:

- You need to track **connected components** in a graph as edges are added.
- You need to detect **cycles** in an undirected graph efficiently.
- You need to answer **"are these two elements in the same set?"** queries dynamically.
- You need to merge sets dynamically.

Common Trigger Phrases

- "Check if adding an edge creates a cycle"
- "Find the number of connected components"
- "Are two nodes in the same component?"
- "Union of sets"
- "Dynamic connectivity"
- "Merge groups"
- "Kruskal's algorithm"
- "Number of provinces / friend circles"
- "Accounts merge"
- "Redundant connection"

4. When Not to Use It

Situation	Alternative
Need to find path between two nodes	Use BFS/DFS or Union-Find with path tracking (not standard DSU).
Need to find the actual path, not just connectivity	DSU only tells if two nodes are connected, not the path. Use BFS/DFS.
Graph is directed	DSU works for undirected connectivity. For directed, use Tarjan's SCC or Kosaraju.

Situation	Alternative
Need to find connected components dynamically with deletions	DSU only supports insertions (union). For deletions, use Link-Cut Tree or offline DSU with rollback.
Need to know component size frequently	Extend DSU with <code>size[]</code> array. Still use DSU.
Need to find which specific elements are in a component	DSU can track size, but listing all elements is $O(n)$ per component. Use DFS for that.

5. Core Concepts

5.1 Parent Array

An array `parent[]` where `parent[i]` points to the parent of `i`. If `parent[i] == i`, then `i` is the root (representative) of its set.

- **Why it matters:** The root identifies the component. Two elements are in the same component iff they have the same root.

5.2 Path Compression

During `find(x)`, set `parent[x] = find(parent[x])` so that `x` points directly to the root.

- **Why it matters:** Makes future `find(x)` calls $O(1)$. Without it, the tree can become a chain and `find(x)` becomes $O(n)$.

5.3 Union by Rank / Size

- **Union by rank:** Attach the tree with smaller rank (height) under the tree with larger rank.
- **Union by size:** Attach the smaller tree (by number of elements) under the larger tree.
- **Why it matters:** Keeps tree depth $O(\log n)$. With path compression, the effective depth is $O(\alpha(n))$.

5.4 Inverse Ackermann Function $\alpha(n)$

The amortized time per operation. For $n \leq 10^6$, $\alpha(n) \leq 5$. For $n \leq 10^{265}$, $\alpha(n) \leq 5$. It's essentially constant.

- **Why it matters:** You can treat DSU operations as $O(1)$ in practice.

6. Step-by-Step Algorithm

DSU Operations

Initialization:

1. For each element `i` from 0 to `n-1`: `parent[i] = i`, `rank[i] = 0`.

`find(x)`:

- 1. If `parent[x] == x`, return `x`.
- 2. Otherwise, `parent[x] = find(parent[x])` (path compression).
- 3. Return `parent[x]`.

union(x, y):

- 1. `rx = find(x), ry = find(y)`.
- 2. If `rx == ry`, return (already in same set).
- 3. If `rank[rx] < rank[ry]`, swap so `rx` is the larger rank.
- 4. `parent[ry] = rx`.
- 5. If `rank[rx] == rank[ry]`, increment `rank[rx]` by 1.

7. Dry Run

Initial: `n = 6`. `parent = [0, 1, 2, 3, 4, 5]`, `rank = [0, 0, 0, 0, 0, 0]`.

Operation	find(0)	find(1)	Action	parent[] (after)	rank[] (after)
union(0,1)	0	1	rank same, attach 1→0	[0,0,2,3,4,5]	[1,0,0,0,0,0]
union(2,3)	2	3	rank same, attach 3→2	[0,0,2,2,4,5]	[0,0,1,0,0,0]
union(0,2)	0	2	rank(0)=1 > rank(2)=1, attach 2→0	[0,0,0,2,4,5]	[1,0,0,0,0,0]
find(3)	—	—	Path compression: parent[3]=find(2)=0	[0,0,0,0,4,5]	[1,0,0,0,0,0]
find(1)	—	—	parent[1]=0, returns 0	—	—

Final state: All of {0,1,2,3} are in one component (root=0). {4} and {5} are separate.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
public:
    vector<int> parent, rank, sz;

    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
```

```

        parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;

        // union by rank (or size)
        if (rank[rx] < rank[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        if (rank[rx] == rank[ry]) rank[rx]++;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    int componentSize(int x) {
        return sz[find(x)];
    }

    int countComponents() {
        int cnt = 0;
        for (int i = 0; i < (int)parent.size(); i++)
            if (parent[i] == i) cnt++;
        return cnt;
    }
};

// Example usage for Kruskal's
int kruskalMST(int n, vector<vector<int>>& edges) {
    // edges: {u, v, w}
    sort(edges.begin(), edges.end(), [](auto& a, auto& b) {
        return a[2] < b[2];
    });

    DSU dsu(n);
    int totalWeight = 0;
    int edgesTaken = 0;

    for (auto& e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (!dsu.same(u, v)) {
            dsu.unite(u, v);
            totalWeight += w;
            edgesTaken++;
            if (edgesTaken == n - 1) break;
        }
    }
    return (edgesTaken == n - 1) ? totalWeight : -1; // -1 if disconnected
}

```

9. Python Implementation

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.size = [1] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def unite(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        # union by rank
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        self.size[rx] += self.size[ry]
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1

    def same(self, x, y):
        return self.find(x) == self.find(y)

    def component_size(self, x):
        return self.size[self.find(x)]

    def count_components(self):
        return sum(1 for i in range(len(self.parent)) if self.parent[i] == i)

# Example: Kruskal MST
def kruskal_mst(n, edges):
    # edges: list of (u, v, w)
    edges.sort(key=lambda e: e[2])
    dsu = DSU(n)
    total_weight = 0
    edges_taken = 0

    for u, v, w in edges:
        if not dsu.same(u, v):
            dsu.unite(u, v)
            total_weight += w
            edges_taken += 1
            if edges_taken == n - 1:
                break
```

```
return total_weight if edges_taken == n - 1 else -1
```

10. Code Explanation

DSU Class — Key Parts

Constructor:

- Initializes `parent[i] = i` (each element is its own root).
- `rank[i] = 0` (initial height of each tree is 0).
- `size[i] = 1` (each component has 1 element).

`find(x)`:

- Base case: if `parent[x] == x`, return `x`.
- Recursive case: `parent[x] = find(parent[x])` — path compression.
- This flattens the tree, making future finds $O(1)$.

`unite(x, y)`:

- Find roots of both elements.
- If same, do nothing.
- Attach the smaller rank tree under the larger rank tree.
- If ranks are equal, rank increases by 1 (the tree height grows by 1).
- Also updates `size[]` for the new root.

`same(x, y)`: Convenience method to check if two elements are in the same component.

`componentSize(x)`: Returns the size of the component containing `x`.

`countComponents()`: Counts how many roots exist (elements where `parent[i] == i`).

Kruskal Integration

- Sort edges by weight.
- For each edge, if `u` and `v` are in different components, unite them and add weight.
- Early stop when `V-1` edges are taken.
- Return -1 if graph is disconnected.

11. Complexity Analysis

Operation	Without Optimization	With Path Compression	With Both Optimizations
<code>find(x)</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized
<code>unite(x,y)</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized
<code>same(x,y)</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized

Overall for Kruskal: $O(E \log E + E \alpha(V)) = O(E \log V)$ since $\alpha(V)$ is negligible.

Space: $O(V)$ for parent, rank, size arrays.

12. Common Patterns

Pattern 1: Cycle Detection in Undirected Graph

When: Adding edges one by one, check if they create a cycle.

Approach: Before adding edge (u,v) , check if `find(u) == find(v)`. If yes, edge creates a cycle.

Pattern 2: Number of Connected Components

When: Count distinct components after processing edges.

Approach: `countComponents()` returns number of roots.

Pattern 3: Dynamic Connectivity (Add-Only)

When: Add edges over time, answer connectivity queries.

Approach: DSU with path compression. Answer `same(u,v)` queries in $O(\alpha(n))$.

Pattern 4: Kruskal's MST

When: Minimum spanning tree.

Approach: Sort edges, DSU for cycle detection.

Pattern 5: Redundant Connection

When: Find the edge that, when added to a graph, first creates a cycle.

Approach: Process edges in order. First edge where `same(u,v)` is true is the redundant one.

Pattern 6: DSU with Rollback

When: Need to undo unions (offline connectivity queries).

Approach: Store changes on a stack. Can't use path compression (breaks rollback). Use union by size only.

13. Common Mistakes

Mistake	Explanation
Forgetting path compression	Without it, find is $O(n)$ per call. DSU becomes useless.
Using path compression with rollback	Path compression modifies parent pointers non-linearly. Can't rollback cleanly.

Mistake	Explanation
Not using union by rank/size	Without it, tree can become a chain, making find $O(n)$.
Swapping incorrectly in unite	Must attach smaller to larger. Wrong swap direction worsens the tree.
Not checking <code>rx == ry</code> in unite	Uniting same set wastes time and can corrupt size/rank.
Using DSU for directed graphs	DSU merges undirected components. For directed SCC, use Tarjan/Kosaraju.
Recursion depth in find	If path compression isn't used, recursion depth can be $O(n)$. Use iterative find or increase recursion limit.
Off-by-one in array size	Ensure DSU is initialized with <code>n</code> elements. If vertices are 1-indexed, use <code>n+1</code> .

14. Edge Cases

Edge Case	What to Expect
<code>n = 0</code>	DSU with 0 elements. No operations.
<code>n = 1</code>	Single element. <code>find(0)</code> returns 0.
All elements in one component	After union operations, all roots are the same.
No unions performed	Each element is its own component.
1-indexed vertices	Initialize DSU with <code>n+1</code> elements. Ignore index 0.
Large <code>n</code> (10^6)	DSU handles easily. $O(\alpha(n))$ per operation.
Duplicate unions	<code>unite(x,y)</code> when already same $\rightarrow$ no-op.

15. Variations

15.1 DSU with Size Tracking

What changes: Maintain `size[rx]` = number of elements whose root is `rx`.

When used: Need to know component size, or use union by size instead of rank.

Importance: High. Common in competitive programming.

15.2 DSU with Rollback (Persistent DSU)

What changes: Store changes on a stack. `unite()` pushes (changed node, old parent, old rank/size). `rollback()` pops and restores.

When used: Divide and conquer on queries (offline algorithms). No path compression.

Importance: Medium. Advanced CP technique.

15.3 DSU with DSU on Tree / DSU on Euler Tour

What changes: Used for subtree queries on trees.

When used: Small-to-large merging on trees.

Importance: Low for interviews. Advanced CP.

15.4 DSU with Difference (Weighted DSU)

What changes: Each node stores a value relative to its parent. `find()` computes the sum/difference along the path.

When used: Problems like "relative ordering" or "find if two values are consistent" (e.g., leetCode 399 — Evaluate Division).

Importance: Medium. Appears in some interview problems.

16. Related Algorithms / Data Structures

Algorithm/DS	Connection	When to Choose
DFS/BFS	Also finds connected components.	DSU is dynamic (add edges). DFS is static (given full graph).
Tarjan's SCC	For directed graph connectivity.	DSU is for undirected. Tarjan finds strongly connected components.
Link-Cut Tree	Dynamic tree connectivity with edge deletions.	Overkill for most problems. Use DSU for add-only.
Segment Tree	DSU with rollback can be used with segment tree over time for offline queries.	Advanced technique for dynamic connectivity offline.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Redundant Connection	LeetCode 684	First edge creating a cycle	Easy
Number of Provinces	LeetCode 547	Count connected components	Easy

Medium

Problem	Platform	Pattern	Difficulty
Accounts Merge	LeetCode 721	DSU to merge accounts + string mapping	Medium
Evaluate Division	LeetCode 399	Weighted DSU with relative values	Medium
Satisfiability of Equality Equations	LeetCode 990	DSU with constraints	Medium

Hard

Problem	Platform	Pattern	Difficulty
Number of Islands II	LeetCode 305	Dynamic DSU (add land cells)	Hard
Checking Existence of Edge Length Limited Paths	LeetCode 1697	Offline queries + DSU	Hard
Dynamic Connectivity	Codeforces	DSU with rollback + divide and conquer	Hard

18. Interview Explanation

"DSU, or Union-Find, is a data structure that tracks disjoint sets. It supports two operations in nearly constant time — `find(x)` returns the representative of the set `x` belongs to, and `union(x, y)` merges the two sets.

Two optimizations make it fast: **path compression**, where we flatten the tree during find, and **union by rank**, where we attach the smaller tree under the larger one. Together, these give amortized $O(\alpha(n))$ per operation, which is essentially constant.

I'd use DSU for cycle detection in Kruskal's algorithm, counting connected components, or dynamic connectivity problems where edges are added over time. The key insight is that DSU handles unions efficiently — it's a 'union-first, ask-questions-later' structure."

19. Revision Notes

- **DSU:** Tracks disjoint sets. `find(x)`, `union(x,y)`.
- **Path compression:** `parent[x] = find(parent[x])` during find.
- **Union by rank:** Attach smaller rank under larger rank.
- **Complexity:** $O(\alpha(n))$ per operation — essentially $O(1)$.
- **Key use in Kruskal:** Cycle detection in $O(\alpha(V))$ per edge.
- **Common trap:** Forget path compression → $O(n)$ per find.
- **Weighted DSU:** Store relative weights between nodes for problems like Evaluate Division.
- **DSU with rollback:** No path compression. Store changes on stack.

20. Final Cheat Sheet

DSU CHEAT SHEET

WHEN TO USE:

- Track connected components
- Cycle detection in undirected graph
- Kruskal's MST
- Dynamic connectivity (add-only)
- "Are these two elements in the same set?"

OPERATIONS:

`find(x)` → $O(\alpha(n))$ → returns root

`unite(x,y)` → $O(\alpha(n))$ → merges sets

`same(x,y)` → $O(\alpha(n))$ → checks if same set

KEY CODE:

```
int find(int x) {
    return parent[x] == x ? x : (parent[x] = find(parent[x]));
}

void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return;
    if (rank[rx] < rank[ry]) swap(rx, ry);
    parent[ry] = rx;
    if (rank[rx] == rank[ry]) rank[rx]++;
}
```

EDGE CASES:

- $n=0$ → no elements
- 1-indexed → initialize with $n+1$
- No unions → each element is its own component

COMMON TRAPS:

- Forgetting path compression
- Not using union by rank
- Using DSU for directed graphs
- Path compression with rollback (breaks it)

SECOND BEST MINIMUM SPANNING TREE (SECOND MST)

1. Overview

A **Second Best Minimum Spanning Tree (Second MST)** is a spanning tree whose total weight is the **second smallest** among all spanning trees of the graph. If the MST is unique, the second MST has weight strictly greater than the MST. If there are multiple MSTs (equal-weight edges), the second MST may have the same weight as the MST.

Essentially: "What's the next cheapest way to connect all nodes?"

2. Intuition

Simple Explanation

The MST is the cheapest way to connect all cities. But what if one of those roads is blocked? The Second MST is the cheapest way to connect all cities **without using one specific MST edge**.

The idea: Take the MST, then for each edge in the MST, remove it and find the cheapest replacement edge that reconnects the graph.

Analogy

Budget airline analogy: You found the cheapest set of flights that connects all cities (MST). But one of those flights gets cancelled. What's the cheapest alternative set of flights? You'd replace that cancelled flight with the next cheapest flight that connects the two regions.

Why It Works

Key observation: The Second MST differs from the MST by exactly **one edge swap**. You remove one MST edge e and add one non-MST edge f that reconnects the graph. The total weight changes by $\text{weight}(f) - \text{weight}(e)$. To get the second best, you choose the swap that minimizes this increase.

Why exactly one swap? Any spanning tree can be transformed into another spanning tree by a sequence of edge swaps. The second best differs in at least one edge. If it differed in two or more, you could improve it by swapping closer to the MST. So the minimal difference is exactly one swap.

3. When to Use It

Use Second MST when:

- Problem asks for "second minimum spanning tree" or "second best".
- Problem asks for "alternative minimum spanning tree".
- Problem asks for "spanning tree with second smallest weight".
- Problem asks about "what if this MST edge is not available".

Common Trigger Phrases

- "Second minimum spanning tree"
 - "Second best"
 - "Alternative spanning tree"
 - "If one edge of MST is removed"
 - "Next cheapest way to connect"
 - "Minimum spanning tree excluding certain edges"
-

4. When Not to Use It

Situation	Alternative
Need just MST	Don't compute Second MST. It's unnecessary work.
Need k-th MST ($k > 2$)	Use more general (and complex) algorithms.
Graph is small ($V \leq 20$)	Use brute force over all spanning trees.
Graph is very dense and MST is unique	Second MST is still useful, but consider if $O(E \log V + E \alpha(V))$ is acceptable.
Need only the weight, not the edges	Same algorithm works. Just track minimum increase.

5. Core Concepts

5.1 Edge Swap

The fundamental operation: remove one MST edge, add one non-MST edge to reconnect the graph.

- **Why it matters:** The second MST is always reachable by exactly one edge swap from the MST.

5.2 Maximum Edge on Path

For a non-MST edge (u, v, w) , the cycle it creates with the MST contains some MST edges. The maximum weight MST edge on the path from u to v in the MST is the best candidate for replacement.

- **Why it matters:** To minimize the weight increase, we want to replace the heaviest MST edge on the cycle.

5.3 LCA + Binary Lifting

To find the maximum weight edge on the path between u and v in the MST efficiently, we can use **Binary Lifting** on the MST tree.

- **Why it matters:** $O(\log V)$ per query instead of $O(V)$. Essential for large graphs.

6. Step-by-Step Algorithm

Approach 1: Simple ($O(E \log E + E * V)$)

1. Find MST using Kruskal. Store MST edges.
2. Initialize `secondBestWeight = INF`.
3. For each MST edge e :
 - Temporarily remove e from the graph.
 - Run Kruskal again (or Prim) on the remaining edges.
 - If a spanning tree exists ($V-1$ edges found), update `secondBestWeight = min(secondBestWeight, newWeight)`.
4. Output `secondBestWeight`.

Approach 2: Efficient ($O(E \log E + E \log V)$)

1. Find MST using Kruskal. Store MST edges and build MST adjacency.
 2. Preprocess MST for LCA with max edge on path (binary lifting).
 3. Initialize `secondBestWeight = INF`.
 4. For each **non-MST edge** (`u, v, w`):
 - Find the maximum weight MST edge `maxW` on the path from `u` to `v` in the MST.
 - The weight of the alternative spanning tree = `MST_weight + w - maxW`.
 - Update `secondBestWeight = min(secondBestWeight, MST_weight + w - maxW)`.
 5. Output `secondBestWeight`.
-

7. Dry Run

Graph: Same as MST dry run example.

```
Vertices: 0, 1, 2, 3, 4
Edges:
0-1: 2
0-3: 6
1-2: 3
1-3: 8
1-4: 5
2-4: 7
3-4: 9
```

MST (from earlier): (0-1:2), (1-2:3), (1-4:5), (0-3:6). Total = 16.

Non-MST edges:

- (2-4:7) — path in MST: 2→1→4. Max edge on path = $\max(3, 5) = 5$. New weight = $16 + 7 - 5 = \mathbf{18}$.
- (1-3:8) — path in MST: 1→0→3. Max edge on path = $\max(2, 6) = 6$. New weight = $16 + 8 - 6 = \mathbf{18}$.
- (3-4:9) — path in MST: 3→0→1→4. Max edge on path = $\max(6, 2, 5) = 6$. New weight = $16 + 9 - 6 = \mathbf{19}$.

Second MST weight: $\min(18, 18, 19) = \mathbf{18}$.

8. C++ Implementation

Efficient Approach (LCA + Binary Lifting)

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;

struct Edge {
    int u, v, w, id;
};
```

```

class DSU {
public:
    vector<int> parent, rank;
    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) swap(rx, ry);
        parent[ry] = rx;
        if (rank[rx] == rank[ry]) rank[rx]++;
    }
};

class LCA {
public:
    int LOG;
    vector<vector<pair<int, int>>> adj;
    vector<vector<int>> up, maxW;
    vector<int> depth;

    LCA(int n, vector<vector<pair<int, int>>>& tree) {
        LOG = ceil(log2(n)) + 1;
        adj = tree;
        up.assign(n, vector<int>(LOG, -1));
        maxW.assign(n, vector<int>(LOG, 0));
        depth.assign(n, 0);
        dfs(0, -1, 0);
    }

    void dfs(int u, int p, int d) {
        depth[u] = d;
        up[u][0] = p;
        for (int j = 1; j < LOG; j++) {
            if (up[u][j-1] != -1) {
                up[u][j] = up[up[u][j-1]][j-1];
                maxW[u][j] = max(maxW[u][j-1], maxW[up[u][j-1]][j-1]);
            }
        }
        for (auto& [v, w] : adj[u]) {
            if (v == p) continue;
            maxW[v][0] = w;
            dfs(v, u, d+1);
        }
    }

    int getMaxOnPath(int u, int v) {

```

```

    int ans = 0;
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int j = 0; j < LOG; j++) {
        if (diff & (1 << j)) {
            ans = max(ans, maxW[u][j]);
            u = up[u][j];
        }
    }
    if (u == v) return ans;
    for (int j = LOG-1; j >= 0; j--) {
        if (up[u][j] != up[v][j]) {
            ans = max(ans, maxW[u][j]);
            ans = max(ans, maxW[v][j]);
            u = up[u][j];
            v = up[v][j];
        }
    }
    ans = max(ans, maxW[u][0]);
    ans = max(ans, maxW[v][0]);
    return ans;
}

};

pair<ll, vector<Edge>> kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
        return a.w < b.w;
    });
    DSU dsu(n);
    vector<Edge> mst;
    ll total = 0;
    for (auto& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            mst.push_back(e);
            total += e.w;
            if ((int)mst.size() == n-1) break;
        }
    }
    return {total, mst};
}

ll secondMST(int n, vector<Edge>& edges, ll mstWeight, vector<Edge>& mst) {
    // Build MST adjacency
    vector<vector<pair<int, int>>> tree(n);
    for (auto& e : mst) {
        tree[e.u].push_back({e.v, e.w});
        tree[e.v].push_back({e.u, e.w});
    }

    // Preprocess LCA for max edge on path
    LCA lca(n, tree);

    ll secondBest = LLONG_MAX;

```

```

set<pair<int,int>> mstEdgeSet;
for (auto& e : mst) {
    mstEdgeSet.insert({min(e.u, e.v), max(e.u, e.v)});
}

for (auto& e : edges) {
    int u = e.u, v = e.v, w = e.w;
    // Skip MST edges
    if (mstEdgeSet.count({min(u,v), max(u,v)})) continue;
    // Max edge on path u-v in MST
    int maxW = lca.getMaxOnPath(u, v);
    ll newWeight = mstWeight - maxW + w;
    if (newWeight > mstWeight) // strictly greater
        secondBest = min(secondBest, newWeight);
    else if (newWeight == mstWeight)
        secondBest = min(secondBest, newWeight); // multiple MSTs case
}
return secondBest;
}

int main() {
    int n = 5;
    vector<Edge> edges = {
        {0, 1, 2, 0}, {0, 3, 6, 1}, {1, 2, 3, 2},
        {1, 3, 8, 3}, {1, 4, 5, 4}, {2, 4, 7, 5}, {3, 4, 9, 6}
    };

    auto [mstWeight, mst] = kruskal(n, edges);
    ll secondBest = secondMST(n, edges, mstWeight, mst);

    cout << "MST Weight: " << mstWeight << "\n";
    cout << "Second MST Weight: " << secondBest << "\n";
    return 0;
}

```

9. Python Implementation

```

import sys
import math
sys.setrecursionlimit(10**6)

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])

```

```

        return self.parent[x]

def unite(self, x, y):
    rx, ry = self.find(x), self.find(y)
    if rx == ry:
        return
    if self.rank[rx] < self.rank[ry]:
        rx, ry = ry, rx
    self.parent[ry] = rx
    if self.rank[rx] == self.rank[ry]:
        self.rank[rx] += 1

def kruskal(n, edges):
    # edges: list of (u, v, w, id)
    edges.sort(key=lambda e: e[2])
    dsu = DSU(n)
    mst = []
    total = 0
    for u, v, w, idx in edges:
        if dsu.find(u) != dsu.find(v):
            dsu.unite(u, v)
            mst.append((u, v, w, idx))
            total += w
            if len(mst) == n - 1:
                break
    return total, mst

class LCA:
    def __init__(self, n, tree):
        self.LOG = math.ceil(math.log2(n)) + 1
        self.tree = tree
        self.up = [[-1] * self.LOG for _ in range(n)]
        self.maxW = [[0] * self.LOG for _ in range(n)]
        self.depth = [0] * n
        self.dfs(0, -1, 0)

    def dfs(self, u, p, d):
        self.depth[u] = d
        self.up[u][0] = p
        for j in range(1, self.LOG):
            if self.up[u][j-1] != -1:
                self.up[u][j] = self.up[self.up[u][j-1]][j-1]
                self.maxW[u][j] = max(self.maxW[u][j-1], self.maxW[self.up[u][j-1]][j-1])
        for v, w in self.tree[u]:
            if v == p:
                continue
            self.maxW[v][0] = w
            self.dfs(v, u, d+1)

    def get_max_on_path(self, u, v):
        ans = 0

```

```

    if self.depth[u] < self.depth[v]:
        u, v = v, u
    diff = self.depth[u] - self.depth[v]
    for j in range(self.LOG):
        if diff & (1 << j):
            ans = max(ans, self.maxW[u][j])
            u = self.up[u][j]
    if u == v:
        return ans
    for j in range(self.LOG - 1, -1, -1):
        if self.up[u][j] != self.up[v][j]:
            ans = max(ans, self.maxW[u][j])
            ans = max(ans, self.maxW[v][j])
            u = self.up[u][j]
            v = self.up[v][j]
    ans = max(ans, self.maxW[u][0])
    ans = max(ans, self.maxW[v][0])
    return ans

def second_mst(n, edges, mst_weight, mst):
    # Build tree adjacency
    tree = [[] for _ in range(n)]
    mst_set = set()
    for u, v, w, idx in mst:
        tree[u].append((v, w))
        tree[v].append((u, w))
        mst_set.add((min(u, v), max(u, v)))

    lca = LCA(n, tree)
    second_best = float('inf')

    for u, v, w, idx in edges:
        if (min(u, v), max(u, v)) in mst_set:
            continue
        max_w = lca.get_max_on_path(u, v)
        new_weight = mst_weight - max_w + w
        if new_weight >= mst_weight:
            second_best = min(second_best, new_weight)

    return second_best

# Example
n = 5
edges = [
    (0, 1, 2, 0), (0, 3, 6, 1), (1, 2, 3, 2),
    (1, 3, 8, 3), (1, 4, 5, 4), (2, 4, 7, 5), (3, 4, 9, 6)
]

mst_weight, mst = kruskal(n, edges)
second = second_mst(n, edges, mst_weight, mst)

```

```
print(f"MST Weight: {mst_weight}")
print(f"Second MST Weight: {second}")
```

10. Code Explanation

LCA with Max Edge on Path — Key Parts

Binary Lifting Table:

- `up[u][j]`: 2^j -th ancestor of `u`.
- `maxW[u][j]`: Maximum edge weight on the path from `u` to its 2^j -th ancestor.

DFS Preprocessing:

- For each node, set `up[u][0] = parent` and `maxW[u][0] = edge weight to parent`.
- For $j > 0$: `up[u][j] = up[up[u][j-1]][j-1]` and `maxW[u][j] = max(maxW[u][j-1], maxW[up[u][j-1]][j-1])`.

getMaxOnPath(u, v):

- Lift `u` to same depth as `v`, tracking max edge weight.
- Lift both nodes together until their ancestors are the same, tracking max edge weight.
- Return the maximum edge weight encountered.

Second MST Computation

- Build MST adjacency from Kruskal output.
- Preprocess LCA for max edge on path.
- For each non-MST edge `(u, v, w)`:
 - Find `maxW` = max edge on path from `u` to `v` in MST.
 - New weight = `mstWeight - maxW + w`.
 - Track minimum among all such swaps.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity	Notes
Simple (remove & recompute)	$O(E \log E + E * V \alpha(V))$	$O(V + E)$	For each MST edge, re-run Kruskal. Too slow for large graphs.
Efficient (LCA)	$O(E \log E + E \log V)$	$O(V \log V + E)$	Sort edges once. LCA gives $O(\log V)$ per non-MST edge.

Breakdown of Efficient Approach:

- Sorting edges: $O(E \log E)$
- Kruskal's MST: $O(E \alpha(V))$
- Building LCA: $O(V \log V)$

- Processing non-MST edges: $O(E \log V)$
- Total: $O(E \log E + E \log V) = O(E \log V)$

12. Common Patterns

Pattern 1: Second Best MST

When: Explicitly asked for "second minimum spanning tree".

Approach: MST + LCA for max edge on path. Swap heaviest MST edge on cycle.

Pattern 2: MST with One Edge Forbidden

When: "If we cannot use a particular edge, what's the MST?"

Approach: If the forbidden edge is not in MST, the MST itself is the answer. If it is in MST, find the best replacement (same as Second MST calculation).

Pattern 3: Critical Edges

When: "Which edges, if removed, increase the MST weight?"

Approach: An MST edge is critical if removing it forces a heavier replacement edge. This is exactly what the Second MST calculation checks.

13. Common Mistakes

Mistake	Explanation
Not considering equal-weight edges	If there are multiple MSTs, second best may have same weight. Check for $\geq$ vs $>$.
Using the simple approach on large graphs	$O(E * V)$ can be too slow for $V > 10^3$. Use LCA approach.
Forgetting disconnected graph	If no spanning tree exists, second MST doesn't exist either.
Wrong max edge on path	Not correctly tracking the maximum edge during LCA lifting.
Not handling the case where graph has only one MST	Second best is then strictly greater.
Overflow in weight calculation	Use <code>long long</code> for large total weights.

14. Edge Cases

Edge Case	What to Expect
Graph with only 1 spanning tree	No second MST exists. Return INF or -1.

Edge Case	What to Expect
Multiple MSTs have same weight	Second MST weight = MST weight (if graph has at least 2 MSTs).
Disconnected graph	No MST exists, so no second MST.
Graph with $V=1$	No spanning tree at all.
Graph with $V=2$	Only one spanning tree (the single edge). No second MST.
Complete graph	Many non-MST edges to check. LCA approach handles it well.

15. Variations

15.1 K-th Minimum Spanning Tree

What changes: Generalize to k-th best. Use Eppstein's algorithm or enumeration.

When used: "Find the k-th cheapest spanning tree."

Importance: Low. Very niche. Rarely appears.

15.2 Minimum Spanning Tree with One Edge Constraint

What changes: "Find MST that must include / must exclude a specific edge."

When used: Problems with forced edges.

Importance: Medium. Good to know.

15.3 Minimum Spanning Tree Under Edge Deletions

What changes: Edges are removed over time, recompute MST after each deletion.

When used: Dynamic graph problems.

Importance: Low. Usually solved with offline queries or recompute from scratch.

16. Related Algorithms / Data Structures

Algorithm/DS	Connection	When to Choose
LCA (Binary Lifting)	Used to find max edge on path efficiently.	Essential for efficient Second MST.
Kruskal / Prim	MST is the foundation.	Run first, then compute Second MST.
DSU	Kruskal's cycle detection.	Used in the MST computation step.
Heavy-Light Decomposition	Alternative to LCA for path max queries.	For more complex tree path queries (sum, min, max, etc.).
Link-Cut Tree	Can handle dynamic MST.	Overkill for static Second MST.

17. Practice Problems

Medium

Problem	Platform	Pattern	Difficulty
Second Minimum Spanning Tree	CSES	Direct Second MST	Medium
MST with One Edge Forbidden	Codeforces (various)	Forbidden edge MST	Medium

Hard

Problem	Platform	Pattern	Difficulty
Find Critical and Pseudo-Critical Edges in MST	LeetCode 1489	Critical edges + Second MST logic	Hard
Minimum Spanning Tree for Each Edge	Codeforces 609E	For each edge, MST that includes it	Hard

18. Interview Explanation

"To find the second best MST, we use the property that the second MST differs from the MST by exactly one edge swap. We remove one MST edge and add the best replacement edge.

A naive approach is: for each MST edge, remove it and run Kruskal again. That's $O(E * V)$, which works for small graphs but not large ones.

The efficient approach uses **LCA with binary lifting** on the MST tree. We preprocess the MST to answer "what's the maximum weight edge on the path between two nodes?" in $O(\log V)$ time. Then for each non-MST edge (u, v, w) , the cycle it creates with the MST has a maximum MST edge. Replacing that heaviest MST edge with (u, v, w) gives the best alternative. We take the minimum among all such alternatives.

The total complexity is $O(E \log E + E \log V)$, which is essentially $O(E \log V)$."

19. Revision Notes

- **Second MST:** One edge swap from MST.
- **Simple approach:** Remove each MST edge $\rightarrow$ re-run Kruskal $\rightarrow O(E * V)$.
- **Efficient approach:** LCA for max edge on path $\rightarrow$ check each non-MST edge $\rightarrow O(E \log V)$.
- **Key formula:** `secondBest = min(secondBest, mstWeight + w_nonMST - maxEdgeOnPath)`.
- **Multiple MSTs:** If equal-weight edges exist, second best can equal MST weight.
- **Edge case:** If graph has only one spanning tree, no second MST exists.

20. Final Cheat Sheet

SECOND MST CHEAT SHEET

WHEN TO USE:

- "Second cheapest spanning tree"
- "Alternative spanning tree"
- "If MST edge is blocked"

KEY IDEA:

Second MST = MST + 1 non-MST edge - 1 MST edge
 Choose swap that minimizes weight increase.

COMPLEXITY:

Simple: $O(E \log E + E * V)$

Efficient: $O(E \log E + E \log V) = O(E \log V)$

KEY FORMULA:

$\text{newWeight} = \text{mstWeight} + w_{\text{nonMST}} - \text{maxEdgeOnPath}$
 $\text{secondBest} = \min(\text{secondBest}, \text{newWeight})$

KEY CODE:

```
// For each non-MST edge (u, v, w):
maxW = lca.getMaxOnPath(u, v) // O(log V)
secondBest = min(secondBest, mstWeight - maxW + w)
```

EDGE CASES:

- Only one spanning tree → no second MST
- Multiple MSTs → second may equal MST weight
- Disconnected graph → no MST

COMMON TRAPS:

- Wrong LCA implementation
- Not handling equal-weight edges properly
- Overflow in total weight

DYNAMIC MINIMUM SPANNING TREE (DYNAMIC MST)

1. Overview

A **Dynamic Minimum Spanning Tree (Dynamic MST)** is a data structure that maintains the MST of a graph as edges are **inserted, deleted, or have their weights updated** over time.

Instead of recomputing the MST from scratch after each change (which costs $O(E \log V)$), a dynamic MST algorithm efficiently updates the MST in $O(\log^2 V)$ or $O(\log^4 V)$ time per operation.

But here's the reality: **fully dynamic MST is extremely complex** and rarely used in interviews or competitive programming. Most problems use:

- **Offline Dynamic MST:** Process all queries offline using divide and conquer + DSU with rollback.
 - **Incremental MST:** Only edge insertions (add edges one by one, recompute or use DSU strategically).
 - **Recompute from scratch:** If the graph is small enough, just re-run Kruskal/Prim.
-

2. Intuition

Simple Explanation

You have a graph that changes over time. Edges are added, removed, or their weights change. You need to know the MST after each change. Recomputing from scratch every time is too slow.

The efficient approach: instead of recomputing everything, only fix the parts that change.

Analogy

City planning analogy: Your city's road network (MST) is the cheapest way to connect all neighborhoods. A new road is built (insertion), an old road collapses (deletion), or a toll changes (update). Instead of redesigning the entire road network from scratch, you only adjust the affected parts.

Why It Works (Offline Approach)

Divide and Conquer on Time: If we know all queries in advance, we can divide the timeline into segments. An edge exists during a contiguous range of time. We build a segment tree over time, insert each edge into the nodes where it's active, and then traverse the tree with DSU + rollback. This avoids implementing complex dynamic data structures.

3. When to Use It

Use Dynamic MST when:

- The graph changes over time (edges added/removed/updated).
- You need the MST after each change.
- Number of changes is large (recomputing from scratch is too slow).
- The problem is **offline** (all queries known in advance).

Common Trigger Phrases

- "After each edge addition, find the MST"
 - "After each edge deletion, find the MST"
 - "Dynamic graph connectivity / MST"
 - "Process queries in order"
 - "Offline queries"
-

4. When Not to Use It

Situation	Alternative
Only edge insertions (no deletions)	Incremental DSU + Kruskal: sort edges once, add edges as they appear.
Small graph ($V, E \leq 100$)	Recompute from scratch after each change.
Small number of queries (≤ 100)	Recompute from scratch.
Online queries (don't know future)	Need fully dynamic MST (Link-Cut Tree). Very complex.
Interview setting	Interviewers rarely ask for full dynamic MST. They'll ask about incremental or recompute approaches.
Need simple solution	Just recompute from scratch and explain the tradeoff.

5. Core Concepts

5.1 Offline vs Online

- **Offline:** All queries are known in advance. We can process them in any order.
- **Online:** Queries come one by one, and we must answer each before seeing the next.
- **Why it matters:** Offline dynamic MST is much easier (divide and conquer + DSU rollback). Online requires Link-Cut Tree.

5.2 Segment Tree Over Time

We build a segment tree where each node represents a time interval. Each edge is active during a contiguous range of time. We insert the edge into all segment tree nodes whose intervals it covers.

- **Why it matters:** This allows us to process all queries in $O((E + Q) \log Q)$ time using DSU with rollback.

5.3 DSU with Rollback

A DSU that supports undoing the last union operation. No path compression (breaks rollback). Only union by size.

- **Why it matters:** When we backtrack in the segment tree traversal, we need to undo the unions we performed.

5.4 Link-Cut Tree (LCT)

A data structure that maintains a forest of trees under edge insertions, deletions, and path queries.

- **Why it matters:** For online dynamic MST, LCT maintains the current MST and supports link/cut operations in $O(\log V)$ amortized.

6. Step-by-Step Algorithm

Offline Dynamic MST (Divide and Conquer + DSU Rollback)

Preprocessing:

1. For each edge, determine the time intervals where it is active.
2. Build a segment tree over the time range $[0, Q-1]$ (Q = number of queries).
3. Insert each edge into the segment tree nodes covering its active intervals.

Traversal: 4. Traverse the segment tree (DFS). 5. When entering a node, apply all unions for edges stored in that node using DSU with rollback. 6. If we are at a leaf (time = t), answer the query at time t (e.g., current MST weight). 7. Recursively process children. 8. When leaving a node, rollback all unions performed in that node.

Incremental MST (Only Insertions)

1. Sort all edges that will ever appear by weight.
2. Maintain DSU of current components.
3. When an edge is added:
 - If `find(u) != find(v)`, add it to MST, union the components.
 - Otherwise, skip it (it would create a cycle with lighter edges).
4. To get current MST weight, maintain a running sum.

7. Dry Run

Incremental MST Example

Initial graph: Empty. $V = 5$.

Edges added in order:

Time	Edge Added	Action	MST Weight	MST Edges
0	(0,1,2)	<code>find(0) != find(1)</code> → add	2	(0-1:2)
1	(1,2,3)	<code>find(1) != find(2)</code> → add	5	(0-1:2), (1-2:3)
2	(0,2,1)	<code>find(0) == find(2)</code> → skip	5	(0-1:2), (1-2:3)
3	(2,3,4)	<code>find(2) != find(3)</code> → add	9	(0-1:2), (1-2:3), (2-3:4)

Note: If edges are added in any order (not sorted), we can't use incremental DSU this way. We need to recompute or use a more complex approach.

8. C++ Implementation

Incremental MST (Edges Added in Order Only)

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
```

```

public:
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }
};

int main() {
    int n = 5;
    vector<tuple<int,int,int>> insertions = {
        {0, 1, 2}, {1, 2, 3}, {0, 2, 1}, {2, 3, 4}
    };

    // Sort by weight (for incremental MST, edges must be processed in sorted
    order)
    // But here we're adding in given order, so we need to process differently.
    // For genuinely incremental (any order), we would need to maintain edge set
    // and recompute or use a dynamic approach.

    // This version: sort once, then process additions.
    sort(insertions.begin(), insertions.end(), [](auto& a, auto& b) {
        return get<2>(a) < get<2>(b);
    });

    DSU dsu(n);
    long long mstWeight = 0;
    vector<tuple<int,int,int>> mst;

    for (auto& [u, v, w] : insertions) {
        if (dsu.find(u) != dsu.find(v)) {
            dsu.unite(u, v);
            mstWeight += w;
            mst.push_back({u, v, w});
        }
    }

    cout << "Final MST Weight: " << mstWeight << "\n";
    return 0;
}

```

DSU with Rollback (for Offline Dynamic MST)

```

#include <bits/stdc++.h>
using namespace std;

struct DSUWithRollback {
    vector<int> parent, sz;
    vector<tuple<int,int,int,int>> history; // (changed_node, old_parent,
    old_root_sz, old_other_root)

    DSUWithRollback(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        // No path compression (rollback wouldn't work)
        while (parent[x] != x) x = parent[x];
        return x;
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) {
            history.push_back({-1, -1, -1, -1}); // placeholder
            return;
        }
        if (sz[rx] < sz[ry]) swap(rx, ry);
        // Save state for rollback
        history.push_back({ry, parent[ry], sz[rx], ry});
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }

    void rollback() {
        auto [node, oldParent, oldSz, otherRoot] = history.back();
        history.pop_back();
        if (node == -1) return;
        // Restore the changed node's parent
        parent[node] = oldParent;
        // Restore the size of the root (which was the larger tree)
        // We need to find the root that had its size increased
        int root = find(otherRoot); // This is the root of the merged tree
        // Actually, the simpler way: store the root that got its size increased
        // Let's redesign: store (root, old_size_of_root, child,
        old_parent_of_child)
        // For simplicity, we'll just store the exact changes
    }
};

// Note: The rollback DSU above is simplified.

```

```
// A proper implementation stores (root, old_size, node, old_parent) for each union.
```

Proper DSU with Rollback

```
#include <bits/stdc++.h>
using namespace std;

struct RollbackDSU {
    vector<int> parent, sz;
    vector<tuple<int,int,int,int>> ops; // (u, old_parent_u, v, old_sz_v)

    RollbackDSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        while (parent[x] != x) x = parent[x];
        return x;
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) {
            ops.push_back({-1, -1, -1, -1});
            return;
        }
        if (sz[rx] < sz[ry]) swap(rx, ry);
        ops.push_back({ry, parent[ry], rx, sz[rx]});
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }

    void rollback() {
        auto [u, old_pu, v, old_sz_v] = ops.back();
        ops.pop_back();
        if (u == -1) return;
        parent[u] = old_pu;
        sz[v] = old_sz_v;
    }

    int snapshot() { return ops.size(); }
    void rollbackTo(int snap) {
        while ((int)ops.size() > snap) rollback();
    }
};
```

9. Python Implementation

Rollback DSU

```
class RollbackDSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.sz = [1] * n
        self.ops = [] # stack of (u, old_parent_u, v, old_sz_v)

    def find(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def unite(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            self.ops.append((-1, -1, -1, -1))
            return
        if self.sz[rx] < self.sz[ry]:
            rx, ry = ry, rx
        self.ops.append((ry, self.parent[ry], rx, self.sz[rx]))
        self.parent[ry] = rx
        self.sz[rx] += self.sz[ry]

    def rollback(self):
        if not self.ops:
            return
        u, old_pu, v, old_sz_v = self.ops.pop()
        if u == -1:
            return
        self.parent[u] = old_pu
        self.sz[v] = old_sz_v

    def snapshot(self):
        return len(self.ops)

    def rollback_to(self, snap):
        while len(self.ops) > snap:
            self.rollback()
```

10. Code Explanation

DSU with Rollback — Key Parts

No Path Compression:

- Path compression modifies parent pointers of many nodes. We can't track all those changes efficiently.
- Without path compression, `find(x)` is $O(\log n)$ (due to union by size).

History Stack:

- Each `unite()` call pushes the changes made onto a stack.
- `rollback()` pops the last operation and restores the state.

snapshot() and rollbackTo():

- `snapshot()` returns the current stack size.
- `rollbackTo(snapshot)` rolls back to the state at that snapshot.
- This is used when traversing the segment tree: take a snapshot before processing a node, rollback to it after.

Segment Tree Over Time (Conceptual)

- Each node of the segment tree corresponds to a time interval $[l, r]$.
- For each edge, we know the time intervals where it's active.
- We insert the edge into $O(\log Q)$ segment tree nodes (the ones that cover its active interval).
- We traverse the tree DFS:
 - At each node, unite all edges stored in that node.
 - If leaf (time = t), answer query for time t .
 - Recurse to children.
 - On exit, rollback all unions.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity	Notes
Recompute from scratch	$O(Q * E \log V)$	$O(V + E)$	Q = queries. Too slow for large Q .
Incremental (only insertions, sorted)	$O(E \log E + Q \alpha(V))$	$O(V + E)$	Only works if edges are added in sorted order.
Offline (segment tree + DSU rollback)	$O((E + Q) \log Q * \alpha(V))$	$O((E + Q) \log Q + V)$	Handles deletions too.
Fully dynamic (Link-Cut Tree)	$O(\log^2 V)$ per operation	$O(V + E)$	Very complex. Online.

Offline approach breakdown:

- Each edge is active over a contiguous time interval. It's inserted into $O(\log Q)$ segment tree nodes.
- Total segment tree insertions: $O(E \log Q)$.
- Each edge is processed once per segment tree node: $O(E \log Q)$ unions.
- Each union is $O(\alpha(V))$ with rollback DSU.
- Total: $O((E + Q) \log Q * \alpha(V))$.

12. Common Patterns

Pattern 1: Incremental MST (Edges Added Over Time)

When: Edges are only added, never removed.

Approach: If edges are added in sorted order, use DSU greedily. If not, maintain a set of edges and recompute MST after each addition (or use a more sophisticated approach).

Pattern 2: Offline Dynamic MST (Edges Added and Deleted)

When: Edges are added and removed over time. All queries known in advance.

Approach: Segment tree over time + DSU with rollback.

Pattern 3: MST After Edge Weight Updates

When: Edge weights change over time.

Approach: Treat each edge's weight as a constant over a time interval. Use the same offline segment tree approach.

13. Common Mistakes

Mistake	Explanation
Using path compression in rollback DSU	Breaks rollback. Use union by size only.
Not saving enough state for rollback	Must save the exact changes made to parent and size arrays.
Forgetting to handle disconnected graphs	Dynamic MST on disconnected graphs doesn't have a spanning tree.
Overcomplicating when incremental is enough	If only insertions, use incremental DSU. Don't implement full dynamic MST.
Using dynamic MST in an interview	Unless the interviewer explicitly asks for it, recompute from scratch or use incremental.
Not handling the case where edge doesn't exist at deletion time	Edge must exist before it can be deleted. Track active edges.

14. Edge Cases

Edge Case	What to Expect
Graph never becomes connected	MST doesn't exist at certain times.
All edges added and removed	Graph may become empty.
Same edge added multiple times	Track edge occurrences.
Self-loops	Can never be in MST. Ignore them.

Edge Case	What to Expect
Parallel edges	Only the smallest weight matters.
$Q = 0$ (no queries)	Nothing to process.

15. Variations

15.1 Incremental MST (Edges Added in Any Order)

What changes: Edges are added in arbitrary order, not sorted by weight.

When used: Real-time systems where edges appear naturally.

How to handle: Maintain a set of edges. After each addition, either recompute from scratch or use a dynamic MST algorithm.

15.2 Decremental MST (Edges Removed Only)

What changes: Edges are only removed, never added.

When used: "After each edge removal, find MST."

How to handle: Process queries in reverse order (treat deletions as insertions). This converts it to incremental MST.

15.3 Fully Dynamic MST (Online)

What changes: Both insertions and deletions, online (don't know future queries).

When used: General case.

How to handle: Link-Cut Tree. Very complex.

16. Related Algorithms / Data Structures

Algorithm/DS	Connection	When to Choose
DSU with Rollback	Backbone of offline dynamic MST.	Use for offline dynamic MST.
Segment Tree	Organizes edges over time.	Use with DSU rollback for offline queries.
Link-Cut Tree	Fully dynamic MST online.	For online dynamic MST. Very complex.
Kruskal / Prim	Recompute from scratch.	Use for small graphs or few queries.
DSU	Incremental MST (additions only).	Use when edges are only added (and sorted).

17. Practice Problems

Medium

Problem	Platform	Pattern	Difficulty
Dynamic Graph Connectivity	Codeforces (various)	Offline DSU with rollback	Medium-Hard
MST on Graph with Updates	Codeforces (various)	Edge weight updates, recompute	Medium

Hard

Problem	Platform	Pattern	Difficulty
Dynamic MST	Codeforces	Fully dynamic MST with Link-Cut Tree	Hard
Road Reparation with Updates	Custom (CSES extension)	Offline dynamic MST	Hard

18. Interview Explanation

"For dynamic MST problems, the approach depends on the type of changes.

If only **edge insertions** are involved, I'd use an incremental approach: maintain a DSU, and for each new edge, if it connects two different components, add it to the MST. This works if edges are processed in sorted order.

If **both insertions and deletions** are involved, and all queries are known in advance, I'd use an **offline approach** with a segment tree over time and a DSU with rollback. Each edge is inserted into the segment tree nodes covering its active time interval. Then we traverse the tree DFS, maintaining the current MST with a rollback DSU, and answer queries at the leaves.

For a fully **online** dynamic MST with both insertions and deletions, the standard approach uses a **Link-Cut Tree**, which is quite complex and rarely asked in interviews.

In practice, for most competitive programming problems, the offline approach is sufficient, and for interviews, explaining the incremental approach or the tradeoff of recomputing from scratch is usually acceptable."

19. Revision Notes

- **Dynamic MST:** Maintain MST as graph changes (insertions, deletions, updates).
- **Incremental (insertions only):** DSU if edges come sorted. Otherwise, recompute or maintain edge set.
- **Offline (insertions + deletions):** Segment tree over time + DSU with rollback.
 - No path compression in rollback DSU.
 - Each edge active over a contiguous time interval.
 - Insert into $O(\log Q)$ segment tree nodes.
 - DFS traversal: unite on entry, rollback on exit.
- **Online (fully dynamic):** Link-Cut Tree. Very complex.
- **Complexity (offline):** $O((E + Q) \log Q \alpha(V))$.

- **Key trick for deletions:** Process queries in reverse to convert deletions to insertions.

20. Final Cheat Sheet

DYNAMIC MST CHEAT SHEET														
WHEN TO USE: • Graph changes over time (insert/delete/update edges) • Need MST after each change • Recomputing from scratch is too slow														
APPROACHES: <table> <tr> <th>Type</th><th>Technique</th><th>Complexity</th></tr> <tr> <td>Incremental</td><td>DSU</td><td>$O(E \log E + Q \alpha(V))$</td></tr> <tr> <td>Offline</td><td>SegTree+Rollback</td><td>$O((E+Q) \log Q \alpha(V))$</td></tr> <tr> <td>Online</td><td>Link-Cut Tree</td><td>$O(\log^2 V)$ per op</td></tr> </table>			Type	Technique	Complexity	Incremental	DSU	$O(E \log E + Q \alpha(V))$	Offline	SegTree+Rollback	$O((E+Q) \log Q \alpha(V))$	Online	Link-Cut Tree	$O(\log^2 V)$ per op
Type	Technique	Complexity												
Incremental	DSU	$O(E \log E + Q \alpha(V))$												
Offline	SegTree+Rollback	$O((E+Q) \log Q \alpha(V))$												
Online	Link-Cut Tree	$O(\log^2 V)$ per op												
KEY CODE (Rollback DSU): <pre>void unite(x, y): save state to stack perform union (no path compression) void rollback(): restore from stack</pre>														
KEY TRICK: Process deletions in reverse → incremental														
EDGE CASES: • Graph may stay disconnected → no MST • Self-loops → ignore • Parallel edges → keep minimum weight														
COMMON TRAPS: • Path compression in rollback DSU (breaks it) • Overcomplicating when incremental is enough • Not handling disconnected graphs														

PRIM'S ALGORITHM

1. Overview

Prim's Algorithm is a greedy algorithm that finds the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph. It grows the MST **one vertex at a time** by always adding the smallest edge that

connects a vertex in the tree to a vertex outside the tree.

Think of it as: *"Start anywhere, and always take the cheapest way to bring in a new vertex."*

2. Intuition

Simple Explanation

Imagine you're building a network. You start at any city. Then, you look at all the roads from your current network to cities not yet connected. You pick the cheapest road and add that city. You repeat until all cities are connected.

Analogy

Building a pipeline: You start at a water source. At each step, you look at all the pipes that can connect any connected point to any unconnected point. You pick the cheapest pipe. You repeat until every point has water.

Step-by-Step Reasoning

1. Start with any vertex. It's your "tree" (just one vertex, no edges).
2. Consider all edges from your tree to vertices outside it.
3. Pick the smallest such edge.
4. Add that edge and the new vertex to your tree.
5. Repeat until all vertices are in the tree.

Why It Works

The **cut property** guarantees correctness: the smallest edge crossing any cut belongs to some MST. Prim's algorithm always picks the smallest edge crossing the cut between the current tree and the rest of the graph. Therefore, every edge it picks is safe.

3. When to Use It

Use Prim's algorithm when:

- The graph is **dense** ($E \approx V^2$) — Prim with adjacency matrix runs in $O(V^2)$.
- The graph is given as an **adjacency list** or **matrix**.
- You need the MST and the graph is connected.
- You want a simpler implementation for dense graphs.

Common Trigger Phrases

- "Minimum cost to connect all points"
 - "MST of a complete graph"
 - "Connect all cities with minimum cost"
 - "Given coordinates, find minimum total distance to connect all points"
-

4. When Not to Use It

Situation	Alternative
Graph is sparse ($E \approx V$)	Kruskal ($O(E \log E)$) or Prim ($O(E \log V)$) — both fine.
Edges are given as a list	Kruskal is more natural with edge lists.
Graph is disconnected	Prim will not visit all vertices. Use Kruskal for MST forest.
Need maximum spanning tree	Negate weights or use a max-heap.
Graph is directed	MST is defined for undirected graphs.

5. Core Concepts

5.1 Cut Property

Any partition of vertices into two sets. The smallest edge crossing the cut is in some MST.

- **Why it matters:** Prim's algorithm maintains a cut (tree vs non-tree) and always picks the smallest edge crossing it.

5.2 Priority Queue (Min-Heap)

A data structure that returns the smallest element.

- **Why it matters:** Prim's algorithm needs to efficiently find the smallest edge crossing the cut. A min-heap gives $O(\log V)$ per operation.

5.3 Visited Array

Boolean array `vis[u]` = true if vertex `u` is already in the MST.

- **Why it matters:** Prevents adding the same vertex twice and keeps the "cut" well-defined.

6. Step-by-Step Algorithm

1. Initialize:

- `vis[] = false` for all vertices.
- `pq = empty min-heap of (weight, vertex)`.
- `mstWeight = 0, edgesTaken = 0`.

2. Pick any starting vertex `s`. Push `(0, s)` to heap.

3. While heap is not empty and `edgesTaken < n`:

- Pop `(w, u)` from heap.
- If `vis[u]` is true, continue.
- Mark `vis[u] = true`.
- `mstWeight += w`.
- `edgesTaken += 1`.
- For each neighbor `v` of `u` with edge weight `wt`:

- If `!vis[v]`, push `(wt, v)` to heap.
4. Output `mstWeight`.

7. Dry Run

Graph:

Vertices: 0, 1, 2, 3

Edges:

0-1: 1

0-2: 4

0-3: 3

1-2: 2

2-3: 5

Start at vertex 0.

Step	Pop	vis[]	mstWeight	Heap (after push)
Init	—	{F,F,F,F}	0	(0,0)
1	(0,0)	{T,F,F,F}	0	(1,1), (4,2), (3,3)
2	(1,1)	{T,T,F,F}	1	(2,2), (4,2→skip), (3,3)
3	(2,2)	{T,T,T,F}	3	(5,2→skip), (3,3)
4	(3,3)	{T,T,T,T}	6	—

MST Weight: 6 **MST Edges:** (0-1:1), (1-2:2), (0-3:3)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

// Prim's algorithm for MST
// adj[u] = list of (neighbor, weight)
int prim(int n, vector<vector<pii>>& adj) {
    vector<bool> vis(n, false);
    priority_queue<pii, vector<pii>, greater<pii>> pq; // min-heap
    int mstWeight = 0;
    int edgesTaken = 0;

    pq.push({0, 0}); // start from vertex 0

    while (!pq.empty() && edgesTaken < n) {
```

```

        auto [w, u] = pq.top();
        pq.pop();

        if (vis[u]) continue;

        vis[u] = true;
        mstWeight += w;
        edgesTaken++;

        for (auto& [v, wt] : adj[u]) {
            if (!vis[v]) {
                pq.push({wt, v});
            }
        }
    }

    // If graph is disconnected, edgesTaken < n
    return (edgesTaken == n) ? mstWeight : -1;
}

int main() {
    int n = 4;
    vector<vector<pii>> adj(n);

    adj[0] = {{1, 1}, {2, 4}, {3, 3}};
    adj[1] = {{0, 1}, {2, 2}};
    adj[2] = {{0, 4}, {1, 2}, {3, 5}};
    adj[3] = {{0, 3}, {2, 5}};

    int result = prim(n, adj);
    cout << "MST Weight: " << result << "\n"; // Output: 6
    return 0;
}

```

Prim with Adjacency Matrix (for dense graphs)

```

#include <bits/stdc++.h>
using namespace std;

int primMatrix(int n, vector<vector<int>>& graph) {
    // graph[u][v] = weight, INF if no edge
    const int INF = 1e9;
    vector<int> key(n, INF);    // minimum weight to connect to tree
    vector<bool> vis(n, false);
    key[0] = 0;
    int mstWeight = 0;

    for (int i = 0; i < n; i++) {
        // Find the unvisited vertex with minimum key
        int u = -1;
        for (int v = 0; v < n; v++) {

```

```

        if (!vis[v] && (u == -1 || key[v] < key[u]))
            u = v;
    }

    if (key[u] == INF) return -1; // disconnected
    vis[u] = true;
    mstWeight += key[u];

    // Update neighbors
    for (int v = 0; v < n; v++) {
        if (!vis[v] && graph[u][v] < key[v])
            key[v] = graph[u][v];
    }
}
return mstWeight;
}

```

9. Python Implementation

```

import heapq

def prim(n, adj):
    # adj: list of list of (neighbor, weight)
    vis = [False] * n
    pq = [(0, 0)] # (weight, vertex)
    mst_weight = 0
    edges_taken = 0

    while pq and edges_taken < n:
        w, u = heapq.heappop(pq)
        if vis[u]:
            continue
        vis[u] = True
        mst_weight += w
        edges_taken += 1

        for v, wt in adj[u]:
            if not vis[v]:
                heapq.heappush(pq, (wt, v))

    return mst_weight if edges_taken == n else -1

# Example
n = 4
adj = [[] for _ in range(n)]
edges = [(0, 1, 1), (0, 2, 4), (0, 3, 3), (1, 2, 2), (2, 3, 5)]
for u, v, w in edges:
    adj[u].append((v, w))

```

```
adj[v].append((u, w))

print(prim(n, adj)) # 6
```

10. Code Explanation

Standard Prim (Heap) — Key Parts

Priority Queue:

- `priority_queue<pii, vector<pii>, greater<pii>>` creates a min-heap.
- Each entry is `(weight, vertex)` — the weight of the edge connecting this vertex to the tree.

Main Loop:

- Pop the smallest weight edge from the heap.
- If the vertex is already visited, skip (it was added via a smaller edge earlier).
- Otherwise, mark visited, add weight to MST, and push all its neighbors' edges.

Stopping condition:

- `edgesTaken < n` ensures we stop when all vertices are in the tree.
- If heap empties before that, the graph is disconnected.

Matrix Prim — Key Parts

- `key[v]` = minimum weight of any edge connecting `v` to the current tree.
- Each iteration: find the unvisited vertex with minimum key, add it to tree, update neighbors.
- No heap needed. $O(V^2)$ per iteration, total $O(V^2)$.

11. Complexity Analysis

Implementation	Time Complexity	Space Complexity	Best For
Prim + Binary Heap	$O(E \log V)$	$O(V + E)$	Sparse graphs
Prim + Adjacency Matrix	$O(V^2)$	$O(V^2)$	Dense graphs
Prim + Fibonacci Heap	$O(E + V \log V)$	$O(V + E)$	Theoretical, rarely used

Breakdown (Heap version):

- Each vertex is pushed once: $O(V)$ pushes.
- Each edge is considered once for push: $O(E)$ pushes.
- Total heap operations: $O((V + E) \log V) = O(E \log V)$ for connected graphs.
- Space: $O(V)$ for arrays + $O(E)$ for adjacency list.

12. Common Patterns

Pattern 1: MST on Complete Graph (Points)

When: Given N points with coordinates, connect all with minimum total distance.

Approach: Use Prim with $O(V^2)$ matrix approach. Build distance matrix on the fly.

Example: LeetCode 1584 — Min Cost to Connect All Points.

Pattern 2: MST with Given Edge List

When: Graph is given as edge list, but Prim is still usable.

Approach: Build adjacency list from edges, run Prim.

Pattern 3: MST with Constraints (Must include certain vertices)

When: Some vertices must be connected first.

Approach: Start Prim from any of those vertices. Or run Prim, then force-include missing edges.

13. Common Mistakes

Mistake	Explanation
Not checking <code>vis[u]</code> before processing	Same vertex can be pushed multiple times. Without check, weight is counted multiple times.
Using <code>vis[v]</code> check before pushing to heap	It's correct but misses potential better edges. Only check on pop.
Using max-heap instead of min-heap	<code>priority_queue<int></code> is max-heap by default. Use <code>greater<pii></code> .
Not handling disconnected graphs	Prim will not visit all vertices. Check <code>edgesTaken == n</code> .
Forgetting to add both directions	Graph is undirected. Add edges in both directions in adjacency list.
Using INF too small	INF should be larger than any possible weight. Use <code>1e9</code> or <code>INT_MAX/2</code> .

14. Edge Cases

Edge Case	What to Expect
<code>n = 0</code>	No vertices. MST weight = 0.
<code>n = 1</code>	Single vertex. MST weight = 0.
Disconnected graph	Return -1 or handle specially.
All edges same weight	Many valid MSTs. Prim picks one based on starting vertex.

Edge Case	What to Expect
Negative weights	Works fine.
Self-loops	Pushed to heap but never added because <code>vis[u]</code> is already true.
Parallel edges	Heap will have both. The smaller one will be processed first.

15. Variations

15.1 Prim with Fibonacci Heap

What changes: Use Fibonacci heap for $O(E + V \log V)$.

When used: Theoretical improvement. Rarely needed in practice.

Importance: Low.

15.2 Prim for Maximum Spanning Tree

What changes: Use max-heap instead of min-heap.

When used: When you need the maximum total weight spanning tree.

Importance: Low. Easy to adapt.

15.3 Prim with Early Termination

What changes: Stop when all vertices are visited, even if heap is not empty.

When used: Always done. Standard.

16. Related Algorithms / Data Structures

Algorithm/DS	Connection	When to Choose
Dijkstra's Algorithm	Code is almost identical. Prim uses edge weight directly; Dijkstra accumulates distances.	Prim: <code>push(wt, v)</code> . Dijkstra: <code>push(dist[u] + wt, v)</code> .
Kruskal's Algorithm	Same goal (MST), different approach.	Kruskal: edge-based, sort edges, DSU. Prim: vertex-based, heap.
BFS	For unweighted graphs, BFS gives a spanning tree.	Use BFS if graph is unweighted.
DSU	Used by Kruskal, not Prim.	Prim doesn't need DSU.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Minimum Spanning Tree	GFG	Direct Prim	Easy
Connecting Cities With Minimum Cost	LeetCode 1135	MST with edge list	Easy

Medium

Problem	Platform	Pattern	Difficulty
Min Cost to Connect All Points	LeetCode 1584	Complete graph, $O(V^2)$ Prim	Medium
Swim in Rising Water	LeetCode 778	MST variant (min max edge)	Hard-Medium

Hard

Problem	Platform	Pattern	Difficulty
Find Critical and Pseudo-Critical Edges in MST	LeetCode 1489	Critical edges with Prim/Kruskal	Hard
Minimum Cost to Make at Least One Valid Path in a Grid	LeetCode 1368	MST on grid	Hard

18. Interview Explanation

"Prim's algorithm finds the MST by growing a tree from a starting vertex. At each step, it picks the smallest edge that connects a vertex in the tree to a vertex outside it.

I implement it using a **min-heap** of `(weight, vertex)`. I start with any vertex, push `(0, start)`, and loop: pop the smallest, if not visited, add it to the tree, and push all its neighbors. The visited check on pop handles duplicates.

The complexity is $O(E \log V)$ with a binary heap, which is efficient for sparse graphs. For dense graphs, I'd use the $O(V^2)$ matrix version instead.

The key difference from Dijkstra is that Prim uses the edge weight directly, not the accumulated distance from the source."

19. Revision Notes

- **Prim:** Greedy, vertex-based, grows tree one vertex at a time.
- **Algorithm:** Start from any vertex → min-heap → pop smallest → if not visited, add, push neighbors.
- **Complexity:** $O(E \log V)$ with heap, $O(V^2)$ with matrix.
- **Key difference from Dijkstra:** Prim uses direct edge weight, Dijkstra uses accumulated distance.
- **Edge case:** Check `edgesTaken == n` for disconnected graphs.
- **Common trap:** Forgetting `vis[]` check → double counting.

20. Final Cheat Sheet

PRIM'S CHEAT SHEET

WHEN TO USE:

- Dense graph ($E \approx V^2$) → use $O(V^2)$ matrix version
- Graph given as adjacency list/matrix
- Need MST of connected graph

COMPLEXITY:

Version	Time	Space
Heap	$O(E \log V)$	$O(V + E)$
Matrix	$O(V^2)$	$O(V^2)$

KEY CODE:

```

pq.push({0, start});
while (!pq.empty()) {
    auto [w, u] = pq.top(); pq.pop();
    if (vis[u]) continue;
    vis[u] = true; mstWeight += w;
    for (auto [v, wt] : adj[u])
        if (!vis[v]) pq.push({wt, v});
}

```

EDGE CASES:

- $n=0$, $n=1$ → weight = 0
- Disconnected → return -1
- Negative weights → works fine

COMMON TRAPS:

- Forgetting vis[] check
- Using max-heap instead of min-heap
- Not handling disconnected graph
- Confusing with Dijkstra (accumulation vs direct weight)

KRUSKAL'S ALGORITHM

1. Overview

Kruskal's Algorithm is a greedy algorithm that finds the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph. It processes edges in **increasing order of weight** and adds an edge to the MST if it connects two different components (i.e., doesn't create a cycle).

Think of it as: "Sort all edges by cost, then pick the cheapest one that doesn't create a loop."

2. Intuition

Simple Explanation

You have a list of all possible roads with their costs. You sort them from cheapest to most expensive. Then you go through the list: if a road connects two areas that aren't already connected, you build it. If they're already connected, you skip it (it would create a useless loop).

Analogy

Building a railway network: You have a list of all possible railway segments with their costs. You sort by cost. You start with the cheapest segment. If it connects two cities that aren't yet connected by rail, you build it. You repeat until all cities are connected. The cheapest segments are prioritized, and you never build a segment that would create a cycle.

Step-by-Step Reasoning

1. Sort all edges by weight (ascending).
2. Initialize each vertex as its own component.
3. For each edge (from smallest to largest):
 - If it connects two different components, add it to MST.
 - Otherwise, skip it.
4. Stop when you have $V-1$ edges.

Why It Works

The **cut property** guarantees correctness: the smallest edge crossing any cut is in some MST. Kruskal's algorithm efficiently maintains components and always considers the smallest edge first. When it finds an edge connecting two components, that edge is the smallest edge crossing the cut between those components, so it's safe to add.

3. When to Use It

Use Kruskal's algorithm when:

- The graph is given as an **edge list**.
- The graph is **sparse** ($E \approx V$).
- You need to sort edges anyway (e.g., for additional processing).
- You need to find the MST of a graph that may be **disconnected** (Kruskal naturally produces a Minimum Spanning Forest).
- You need to find the **maximum spanning tree** (just sort descending).

Common Trigger Phrases

- "Minimum cost to connect all nodes"
- "Given a list of connections with costs"
- "Find the MST"
- "Network design with minimum cost"

4. When Not to Use It

Situation	Alternative
Graph is very dense ($E \approx V^2$)	Sorting all V^2 edges is $O(V^2 \log V)$. Prim with $O(V^2)$ matrix is better.
Graph is given as adjacency matrix	Building edge list from matrix is $O(V^2)$ anyway. Prim's matrix version is simpler.
Need to handle dynamic changes	Kruskal requires re-sorting on each change. Use Prim or dynamic MST.
Graph is directed	MST is defined for undirected graphs.

5. Core Concepts

5.1 Edge List

A list of all edges (u, v, w) in the graph.

- **Why it matters:** Kruskal's algorithm works directly on the edge list. Sorting the edge list is the main operation.

5.2 Disjoint Set Union (DSU)

A data structure that tracks which vertices are in the same component.

- **Why it matters:** Kruskal needs to check if adding an edge creates a cycle. DSU's `find(u) != find(v)` check tells us if two vertices are in different components.

5.3 Cycle Property

In any cycle, the heaviest edge is never in any MST.

- **Why it matters:** Kruskal's algorithm implicitly uses this: when we process edges in increasing order, any edge that creates a cycle is the heaviest edge in that cycle, so it's correctly skipped.

6. Step-by-Step Algorithm

1. Sort all edges by weight (ascending).
2. Initialize DSU with all vertices (each is its own component).
3. Initialize `mstWeight = 0`, `mstEdges = []`, `edgesTaken = 0`.
4. For each edge (u, v, w) in sorted order:
 - If `dsu.find(u) != dsu.find(v)`:
 - Add edge to MST.
 - `mstWeight += w`
 - `mstEdges.push_back((u, v, w))`
 - `dsu.unite(u, v)`
 - `edgesTaken += 1`
 - If `edgesTaken == V-1`, break.

5. If `edgesTaken < V-1`, graph is disconnected. Otherwise, output `mstWeight`.

7. Dry Run

Graph:

```
Vertices: 0, 1, 2, 3, 4
Edges:
0-1: 10
0-2: 6
0-3: 5
1-3: 15
2-3: 4
```

Sorted edges:

Edge	Weight
2-3	4
0-3	5
0-2	6
0-1	10
1-3	15

Step-by-step:

Edge	Weight	find(u)!=find(v)?	Action	Components	edgesTaken
Init	—	—	—	{0},{1},{2},{3},{4}	0
2-3	4	Yes (2≠3)	Add	{0},{1},{2,3},{4}	1
0-3	5	Yes (0≠2)	Add	{0,2,3},{1},{4}	2
0-2	6	No (same component)	Skip	{0,2,3},{1},{4}	2
0-1	10	Yes (0≠1)	Add	{0,1,2,3},{4}	3
1-3	15	No (same component)	Skip	{0,1,2,3},{4}	3

MST: (2-3:4), (0-3:5), (0-1:10) — **Total: 19**

Note: Vertex 4 is disconnected from the graph. The MST only covers 4 vertices.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u, v, w;
};

class DSU {
public:
    vector<int> parent, rank;
    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) swap(rx, ry);
        parent[ry] = rx;
        if (rank[rx] == rank[ry]) rank[rx]++;
    }
};

// Returns (totalWeight, mstEdges)
pair<int, vector<Edge>> kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);
    vector<Edge> mst;
    int totalWeight = 0;

    for (auto& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            mst.push_back(e);
            totalWeight += e.w;
            if ((int)mst.size() == n - 1) break;
        }
    }

    if ((int)mst.size() != n - 1) {
        // Graph is disconnected
        return {-1, {}};
    }
}

```

```

        return {totalWeight, mst};
    }

    int main() {
        int n = 5;
        vector<Edge> edges = {
            {0, 1, 10}, {0, 2, 6}, {0, 3, 5},
            {1, 3, 15}, {2, 3, 4}
        };

        auto [weight, mst] = kruskal(n, edges);
        if (weight == -1) {
            cout << "Graph is disconnected\n";
        } else {
            cout << "MST Weight: " << weight << "\n";
            for (auto& e : mst)
                cout << e.u << " - " << e.v << " : " << e.w << "\n";
        }
        return 0;
    }

```

9. Python Implementation

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def unite(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1

def kruskal(n, edges):
    # edges: list of (u, v, w)
    edges.sort(key=lambda e: e[2])
    dsu = DSU(n)
    mst = []
    total_weight = 0

```

```
for u, v, w in edges:
    if dsu.find(u) != dsu.find(v):
        dsu.unite(u, v)
        mst.append((u, v, w))
        total_weight += w
        if len(mst) == n - 1:
            break

if len(mst) != n - 1:
    return -1, [] # disconnected
return total_weight, mst

# Example
n = 5
edges = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]
weight, mst = kruskal(n, edges)
print(f"MST Weight: {weight}")
print("Edges:", mst)
```

10. Code Explanation

Key Parts

Sorting:

- `sort(edges, [(Edge& a, Edge& b){ return a.w < b.w; }])` — sorts by weight ascending.
- This is the bottleneck of the algorithm: $O(E \log E)$.

Main Loop:

- For each edge (in sorted order):
 - `dsu.find(u) != dsu.find(v)` checks if `u` and `v` are in different components.
 - If yes, adding this edge won't create a cycle. Add it to MST.
 - If no, skip (adding would create a cycle).
- Early break when `mst.size() == n-1` (MST complete).

Disconnected Graph Check:

- After processing all edges, if `mst.size() < n-1`, the graph is disconnected.
- Return -1 or handle appropriately.

11. Complexity Analysis

Operation	Complexity	Notes
Sorting edges	$O(E \log E)$	The dominant factor
DSU operations	$O(E \alpha(V))$	Nearly $O(E)$

Operation	Complexity	Notes
Total	$O(E \log E) = O(E \log V)$	Since $E \leq V^2$, $\log E = O(\log V)$

Space Complexity: $O(V + E)$ for storing edges + DSU arrays.

12. Common Patterns

Pattern 1: Direct MST

When: Given a graph, find the MST.

Approach: Run Kruskal.

Pattern 2: Minimum Spanning Forest

When: Graph may be disconnected. Find MST of each component.

Approach: Run Kruskal. It will naturally stop when edges run out. The MST edges found form the Minimum Spanning Forest.

Pattern 3: Maximum Spanning Tree

When: Need the highest total weight spanning tree.

Approach: Sort edges in descending order. Everything else is the same.

Pattern 4: K-th Smallest Edge in MST

When: Need to find what the k-th smallest edge in the MST is.

Approach: Sort edges, run Kruskal, track the order of addition.

13. Common Mistakes

Mistake	Explanation
Not sorting edges	Kruskal doesn't work without sorting.
Sorting in descending order	That gives maximum spanning tree, not minimum.
Forgetting path compression in DSU	Without it, DSU find is $O(n)$.
Not checking <code>find(u) != find(v)</code>	Adding edges without checking creates cycles.
Not handling disconnected graphs	Kruskal returns $< V-1$ edges. Check <code>mst.size()</code> .
Using 1-indexed vertices without adjusting DSU size	Initialize DSU with <code>n+1</code> if vertices are 1-indexed.

14. Edge Cases

Edge Case	What to Expect
n = 0	No edges. Weight = 0.
n = 1	No edges needed. Weight = 0.
Disconnected graph	Kruskal returns < V-1 edges. Detect and handle.
Graph with parallel edges	Only the smallest weight matters. Sorting handles this.
Graph with self-loops	<code>find(u) == find(u)</code> always true, so they're never added.
All edges same weight	Kruskal picks any V-1 edges that form a spanning tree.
Negative weights	Works fine. They'll be sorted first.

15. Variations

15.1 Maximum Spanning Tree

What changes: Sort descending.

When used: Need maximum total weight spanning tree.

Importance: Low. Easy to adapt.

15.2 Kruskal with Counting Sort

What changes: If weights are small integers, use counting sort for $O(E + \text{maxWeight})$.

When used: Very small weight range.

Importance: Rare. Niche optimization.

15.3 Reverse-Delete Algorithm

What changes: Start with all edges, remove the heaviest edge that doesn't disconnect the graph.

When used: Theoretical interest.

Importance: Low. Rarely used.

16. Related Algorithms / Data Structures

Algorithm/DS	Connection	When to Choose
DSU	Backbone of Kruskal.	Always used with Kruskal.
Prim's Algorithm	Same goal, different approach.	Kruskal: edge-based, good for sparse graphs. Prim: vertex-based, good for dense graphs.

Algorithm/DS	Connection	When to Choose
Borůvka's Algorithm	Another MST algorithm.	Parallel processing. Rarely used.
Dijkstra's Algorithm	Different problem (shortest path).	Kruskal: MST. Dijkstra: shortest path from source.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Minimum Spanning Tree	GFG	Direct Kruskal	Easy
Connecting Cities With Minimum Cost	LeetCode 1135	Direct Kruskal	Easy

Medium

Problem	Platform	Pattern	Difficulty
Road Reparation	CSES	Kruskal + disconnected check	Medium
Find Critical and Pseudo-Critical Edges in MST	LeetCode 1489	Kruskal + edge classification	Hard-Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Spanning Tree for Each Edge	Codeforces 609E	Kruskal + LCA for max edge	Hard
Second Minimum Spanning Tree	CSES	Kruskal + LCA	Hard

18. Interview Explanation

"Kruskal's algorithm finds the MST by sorting all edges by weight and then greedily picking the smallest edge that doesn't create a cycle. I use a **Disjoint Set Union** to detect cycles in near-constant time.

The algorithm is straightforward: sort edges, then for each edge, if it connects two different components, add it to the MST and merge the components. Stop when I have $V-1$ edges.

The complexity is $O(E \log E)$ due to sorting, which is $O(E \log V)$ for connected graphs. The DSU operations are effectively $O(1)$ per edge.

I'd use Kruskal when the graph is given as an edge list, or when the graph is sparse. For dense graphs, I'd prefer Prim's algorithm."

19. Revision Notes

- **Kruskal:** Sort edges → DSU → add smallest non-cycle edge.
- **Complexity:** $O(E \log E)$ time, $O(V + E)$ space.
- **DSU:** `find()` with path compression, `unite()` with union by rank.
- **Edge case:** Disconnected graph → check `mst.size() == V-1`.
- **Variation:** Sort descending for maximum spanning tree.
- **Common trap:** Forgetting to sort edges.

20. Final Cheat Sheet

KRUSKAL'S CHEAT SHEET

WHEN TO USE:

- Graph given as edge list
- Sparse graph ($E \approx V$)
- Need Minimum Spanning Forest (disconnected graph)
- Need maximum spanning tree (sort descending)

COMPLEXITY:

Time: $O(E \log E) = O(E \log V)$

Space: $O(V + E)$

KEY CODE:

```
sort(edges);
for (auto& e : edges) {
    if (dsu.find(e.u) != dsu.find(e.v)) {
        dsu.unite(e.u, e.v);
        mst.push_back(e);
        totalWeight += e.w;
    }
}
```

EDGE CASES:

- $n=0$, $n=1 \rightarrow \text{weight} = 0$
- Disconnected → check `mst.size() == n-1`
- Negative weights → works fine (sorted first)
- Self-loops → never added (same component)

COMMON TRAPS:

- Forgetting to sort
- No path compression in DSU
- Not handling disconnected graphs
- 1-indexed vertices (use $n+1$ in DSU)